

第十九届全国大学生信息安全竞赛（作品赛）
暨第三届”长城杯”网数智安全大赛（作品赛）
作品报告

☐ 命题赛道

☒ 自由赛道

作品名称： FoodShield——外卖订单隐私认证与可信审计系统

电子邮箱： 3863102544@qq.com

提交日期： 2026 年 7 月 3 日

填写说明

1. 所有参赛项目必须为一个基本完整的设计。作品报告书旨在能够清晰准确地阐述（或图示）该参赛队的参赛项目（或方案）。
2. 作品报告采用 A4 纸撰写。除标题外，所有内容必需为宋体、小四号字、1.5 倍行距。
3. 作品报告中各项目说明文字部分仅供参考，作品报告书撰写完毕后，请删除所有说明文字。（本页不删除）
4. 作品报告模板里已经列的内容仅供参考，作者可以在此基础上增加内容或对文档结构进行微调。
5. 为保证网评的公平、公正，作品报告中应避免出现作者所在学校、院系和指导教师等泄露身份的信息。

摘要

随着外卖平台即时通信功能的广泛使用，用户与骑手之间的订单沟通在提升配送效率的同时，也带来了**身份信息暴露、订单隐私泄露、聊天记录篡改以及纠纷追责困难**等安全问题。如何在保护用户隐私的同时保留必要的审计与溯源能力，是一个具有实际应用价值的信息安全问题。

针对上述问题，本作品设计并实现了面向外卖订单通信的隐私认证与可信审计系统 FoodShield。系统以“**日常匿名、异常可溯源**”为设计原则，在不暴露用户真实身份的前提下完成订单认证与实时通信，并通过密码学手段保障通信日志的可信性。当发生投诉或纠纷时，管理员可在权限约束下基于订单号进行条件溯源，在隐私保护与平台管理之间形成平衡。

本作品的主要成果如下：

1. 面向外卖订单场景设计了可控匿名通信模型。日常通信中隐藏用户真实身份，异常场景下支持审批式条件溯源，避免了完全匿名不可管理与直接实名过度暴露两类问题。
2. 构建了基于国密算法的订单认证与消息保护机制。通信参与方在不暴露真实身份的情况下证明订单合法性，消息明文在存储阶段经过加密处理，日志篡改可被密码学手段检测。
3. 实现了基于 Merkle Tree 的通信日志可信审计。通过快照比对验证历史通信记录的完整性，为订单纠纷处理提供可验证的电子证据。

关键词：外卖平台；可控匿名；订单认证；Merkle Tree；条件溯源；可信审计

目 录

摘要	5
第一章 作品概述	7
1.1 背景分析	7
1.1.1 外卖平台即时通信功能的普及与安全风险	7
1.1.2 外卖订单通信中的四类安全问题	8
1.1.3 国密算法推广与相关政策法规背景	8
1.2 相关工作与现有方案不足	9
1.3 作品目标	10
1.4 我们的工作	11
1.5 应用前景	13
第二章 预备知识	14
2.1 HMAC 消息认证码	14
2.2 SM3 密码杂凑算法	16
2.3 SM4 对称加密算法	17
2.4 PID 匿名身份机制	19
2.5 WebSocket 实时通信机制	20
2.6 Merkle Tree 完整性验证	22
2.7 条件溯源与可控匿名模型	24
第三章 系统设计	26
3.1 系统总体方案	26
3.2 系统参与方与信任边界	27
3.3 威胁模型	29
3.4 安全目标	31
3.5 系统总体架构	31
3.6 数据库设计	34
3.7 用户注册与防机器注册机制	39
3.8 PID 匿名身份生成机制	41
3.9 订单 Token 认证机制	42
3.10 用户端与骑手端匿名通信机制	45
3.11 消息加密存储与哈希摘要机制	46

3.12	Merkle 日志完整性验证机制	48
3.13	条件溯源与管理员审计机制	51
3.14	三端摘要一致性增强机制	53
3.15	方案安全性分析	55
第四章	作品实现	58
4.1	系统开发环境	58
4.2	后端服务实现	59
4.3	用户端功能实现	60
4.4	骑手端功能实现	64
4.5	管理员端功能实现	67
4.6	注册、下单与订单认证流程实现	70
4.7	WebSocket 通信流程实现	71
4.8	Merkle 快照与完整性验证实现	72
4.9	条件溯源流程实现	75
4.10	系统运行界面	78
第五章	作品测试与分析	83
5.1	测试环境	83
5.2	功能测试	83
5.3	安全性测试	84
5.4	日志篡改检测测试	87
5.5	越权访问测试	87
5.6	性能测试	88
5.7	对比分析	90
第六章	系统创新性与应用价值	92
6.1	贴近生活的高频应用场景	92
6.2	面向订单流程的多机制融合	92
6.3	日常匿名、异常可追责的安全模型	93
6.4	用户端、骑手端与管理员端贯通的系统闭环	94
6.5	面向即时配送与平台治理的扩展价值	95
第七章	总结	96
7.1	作品总结	96
7.2	未来工作	97

参考文献	99
A 附录	101
B GitHub 仓库地址	101
C 项目目录结构	101
D 核心模块说明	101
4.1 国密算法模块 (crypto/)	101
4.2 服务端模块 (server/)	102
4.3 前端页面 (frontend/)	102
E 攻击演示脚本说明	102
F 性能测试脚本说明	103
G 环境配置与启动说明	103
7.1 环境要求	103
7.2 安装与启动	103
H 安全配置说明	104
I 推荐演示流程	105

第一章 作品概述

1.1 背景分析

1.1.1 外卖平台即时通信功能的普及与安全风险

近年来，外卖平台已成为我国数字经济中规模最大、渗透率最高的生活服务场景之一。根据中国互联网络信息中心（CNNIC）发布的统计报告，截至 2024 年 12 月，我国网上外卖用户规模已达 5.92 亿人，占网民整体的 53.4%；至 2025 年 12 月，该规模进一步增长至 6.30 亿人，占网民整体的 56.0%。中国饭店协会数据显示，2023 年我国餐饮外卖市场规模约 1.2 万亿元，占餐饮收入的比重升至 22.6%。外卖服务已从单纯的餐饮配送延伸至即时零售、药品配送、生鲜采购等多元场景，成为支撑城市生活运转的重要基础设施。

在庞大的用户基数和业务规模驱动下，外卖平台普遍引入了用户与骑手之间的即时通信（Instant Messaging）功能。该功能允许用户在订单创建后通过平台内置的聊天界面与配送骑手进行实时沟通，用于确认配送地址、沟通特殊需求、反馈配送问题等。然而，即时通信功能在提升配送效率的同时，也引入了新的安全隐患。与传统的电话沟通不同，订单即时通信产生的是可持久化存储的文本消息记录，这些记录中可能包含用户的家庭住址、生活习惯、消费偏好等敏感信息；同时，通信参与方的身份信息、订单信息与通信内容在系统中高度关联，一旦安全防护不足，将导致用户隐私的大面积暴露。

2025 年 4 月，四川成都的王女士在外卖平台购买消毒液和计生用品后，配送骑手竟向其发送包含商品明细的骚扰短信。该事件表明，即便平台声称采用了虚拟号码加密技术，配送员仍可通过技术漏洞获取用户的真实手机号码及订单商品明细，并据此实施骚扰。类似事件并非个案：贵州省龙里县人民法院审结的一起案件中，某快递公司装车员在“每条信息 0.8 元”的利诱下连续偷拍快递面单，将数千条包含姓名、电话、住址及购买物品的隐私信息上传至网盘交易，最终 4 名涉案人员因侵犯公民个人信息罪获刑。上述案例揭示了配送行业长期存在的隐私泄露隐患——在缺乏系统性安全设计的订单通信体系中，用户隐私处于事实上的“裸奔”状态。

1.1.2 外卖订单通信中的四类安全问题

通过对现有外卖平台订单通信流程的分析，并结合上述真实安全事件，我们归纳出外卖订单通信场景中存在的四类核心安全问题：

(1) 身份信息暴露 在传统外卖订单通信模式下，用户身份、订单身份与通信身份往往紧密绑定。骑手端在接单后可以直接获取用户的真实姓名、手机号码、收货地址等敏感信息，部分平台虽采用了虚拟号码技术，但据《法治日报》调查，实际运营中仍存在虚拟号短信通道未加密、多次通话后自动关联真实号码、配送端页面未完全屏蔽敏感字段等技术缺陷。这些缺陷使得骑手能够在配送过程中获取超出业务必要范围的用户个人信息，进而可能引发骚扰、恐吓等二次侵害。

(2) 订单隐私泄露 现有外卖平台的即时通信功能主要关注业务连通性，缺乏对订单参与方合法性的系统化认证机制。在实际场景中，攻击者可能通过伪造订单身份、复用历史订单凭证或利用系统权限漏洞进入非自身订单的通信通道，从而窃取其他用户的通信内容或进行恶意骚扰。由于缺少基于密码学的订单认证机制，平台难以在不暴露用户真实身份的前提下验证通信参与方是否为合法的订单相关方。

(3) 聊天记录篡改 外卖订单通信产生的消息记录在发生配送纠纷、恶意投诉或法律诉讼时具有重要的证据价值。然而，现有平台的通信日志存储于平台中心化数据库中，既缺乏密码学完整性保护机制，也缺乏独立的第三方验证手段。当平台自身或具有数据库访问权限的内部人员篡改、删除或插入消息记录时，用户和监管机构难以发现和证明记录已被修改，从而导致通信记录的证据可信度不足。

(4) 纠纷追责困难 在完全匿名或弱匿名的通信环境中，用户和骑手之间的恶意行为（如骚扰、威胁、虚假投诉等）难以溯源到真实责任主体。如果系统默认暴露真实身份，则又会削弱隐私保护效果，形成“完全匿名不可管理”与“直接实名过度暴露”的两难困境。如何在保护日常通信隐私的同时保留异常情况下的条件溯源能力，是外卖订单通信安全需要解决的关键问题。

1.1.3 国密算法推广与相关法律法规背景

在外卖订单通信安全问题的技术层面，密码算法的选择至关重要。长期以来，RSA、AES、SHA-256 等国际通用密码算法在信息系统中被广泛使用，其安全性经过了较充分的公开分析和工程验证。然而，在我国商用密码推广和信息系统合规建设背景下，单纯依赖国际算法体系，可能在自主可控、国产密码适配、商用密码产品认证以及密码应用安全性评估等方面存在不足。为保障国家信息安全，我国大力推进商用密码算法的自主研发和推广应用。

2019 年 10 月 26 日，第十三届全国人民代表大会常务委员会第十四次会议通过《中华人民共和国密码法》，自 2020 年 1 月 1 日起施行。该法明确将密码分为核心密码、普通密码和商用密码，规定商用密码用于保护不属于国家秘密的信息，鼓励商用

密码技术的研究开发、学术交流、成果转化和推广应用。在密码法的框架下，国家密码管理局陆续发布了 SM 系列商用密码算法标准，其中 SM3 密码杂凑算法（GB/T 32905-2016）输出 256 bit 消息摘要，SM4 分组密码算法（GB/T 32907-2016）采用 128 bit 分组长度和 128 bit 密钥长度，SM2 椭圆曲线公钥密码算法（GB/T 32918）提供了数字签名、密钥交换和公钥加密等功能。这些国密算法已在金融、电力、政务、交通等领域得到广泛应用，为我国信息系统的安全自主可控提供了技术基础。

在个人信息保护层面，2021 年 8 月 20 日通过的《中华人民共和国个人信息保护法》（自 2021 年 11 月 1 日起施行）确立了处理个人信息应遵循的“合法、正当、必要和诚信原则”，并明确提出了“最小必要原则”——处理个人信息应当具有明确、合理的目的，并应当与处理目的直接相关，采取对个人权益影响最小的方式。对于外卖配送场景而言，配送员仅需知晓收件人地址和虚拟号码，商品明细、真实电话号码等均属非必要信息，不应向配送环节披露。此外，《中华人民共和国数据安全法》（自 2021 年 9 月 1 日起施行）要求数据处理者建立健全全流程数据安全管理制度，采取相应的技术措施保障数据安全。国务院颁布的《“十四五”数字经济发展规划》亦提出要加强数据加密技术发展，提升数字经济安全体系的保障能力。

上述法律法规和政策文件为外卖订单通信安全系统的设计提供了明确的合规指引：系统应当在“最小必要原则”的指导下，采用国密算法构建身份匿名、订单认证、消息保护和日志审计机制，在保护用户隐私的同时满足平台管理和纠纷追责的监管要求。

1.2 相关工作与现有方案不足

针对外卖订单通信中的安全问题，学术界和产业界已开展了一定的研究和实践，但现有方案在系统性、安全性和适用性方面仍存在不足。

现有外卖平台的隐私保护措施 以美团、饿了么为代表的外卖平台已推出虚拟号码、隐私面单等隐私保护功能。例如，美团规则中心规定商家和骑手需通过平台生成的虚拟号码联系用户，虚拟号码过期后即无法再行联系。然而，这些措施主要针对电话通信环节的号码脱敏，并未覆盖订单即时通信的全流程。如前文所述，虚拟号码技术在实际部署中存在短信通道未加密、多次通话后关联真实号码等漏洞；同时，现有平台缺乏对聊天消息内容的加密存储、通信日志的完整性验证以及异常行为的条件溯源机制，难以应对日志篡改、越权访问和纠纷追责等安全需求。

通用匿名通信系统 以 Tor (The Onion Router) 为代表的匿名通信系统通过多层加密和中间节点转发实现通信匿名性，在隐私保护领域具有重要影响。然而，这

类系统的设计目标是通用互联网通信的匿名性，并未考虑外卖订单场景中订单认证、配送业务逻辑和平台管理等特殊需求。在 Tor 等系统中，通信参与方完全匿名，平台无法在发生纠纷时进行条件溯源，因此不适合直接应用于外卖订单通信场景。

学术研究现状 在学术研究层面,条件隐私保护(Conditional Privacy-Preserving)和可溯源匿名(Traceable Anonymity)已在车联网、电子投票、移动支付等领域得到较多研究。这些研究通常采用群签名、环签名、盲签名或零知识证明等密码学工具实现匿名性与可溯源性之间的平衡。然而，面向外卖订单即时通信场景的条件隐私保护方案研究尚处于起步阶段，现有方案多数停留在理论分析层面，缺少可运行的系统原型和端到端的安全机制闭环。

综上所述，现有方案在外卖订单通信安全方面存在三个主要不足：一是隐私保护措施碎片化，缺少覆盖身份匿名、消息保护和日志审计的系统化设计；二是匿名机制与平台管理之间缺乏平衡，要么过度暴露身份、要么完全不可溯源；三是多数方案缺少工程实现和实际验证。本作品旨在填补这一空白，设计并实现面向外卖订单通信场景的隐私认证与可信审计系统。

1.3 作品目标

针对上述安全问题与现有方案的不足，本作品旨在设计并实现一种面向外卖平台的隐私认证与可信审计系统——FoodShield。系统的核心安全目标如表1所示。

表 1: FoodShield 系统安全目标

编号	安全目标	定义
目标一	身份匿名性	通信参与方在日常业务过程中无法获取对方真实身份, 仅能看到匿名标识或订单相关信息。
目标二	订单认证性	只有持有合法订单凭证的参与方才能进入对应订单的通信通道, 攻击者无法伪造订单身份。
目标三	消息保密性	通信消息在存储阶段经过加密处理, 管理员和未授权方默认无法查看消息明文。
目标四	日志完整性	通信日志的任何篡改、删除、插入或重排均可被检测, 日志记录具备密码学完整性证明。
目标五	条件可溯源	在投诉或审计场景下, 管理员可在权限控制和审计记录约束下基于订单号恢复必要身份信息。

上述五个安全目标相互支撑, 共同构成了“日常匿名、异常可溯源”的可控匿名安全模型。其中, 目标一和目标二保障日常通信中的隐私保护和认证安全, 目标三和目标四保障通信数据的保密性和可信性, 目标五则在异常情况下为平台管理和纠纷处理提供追责能力。

1.4 我们的工作

为实现上述安全目标, 我们设计并实现了 FoodShield 系统。该系统以国密算法为核心, 围绕外卖订单通信场景构建了从匿名身份到可信审计的完整安全机制闭环。系统的技术结构主要包括四个层次:

- **匿名身份层:** 通过基于 HMAC-SM3 的 PID 生成机制, 将用户真实身份与通信身份分离, 实现日常通信中的身份匿名。
- **订单认证层:** 通过基于 HMAC-SM3 的订单 Token 机制, 验证通信参与方的订单合法性, 防止伪造订单身份进入通信通道。
- **安全通信层:** 通过 WebSocket 实现订单级实时通信, 结合 SM4-CBC 加密存储

和 SM3 消息哈希，保障消息内容的保密性和可验证性。

- **可信审计层：**通过 Merkle Tree 日志完整性验证和条件溯源机制，实现通信日志的密码学完整性证明和异常情况下的受控溯源。

具体而言，我们的主要工作如下：

(1) 基于 HMAC-SM3 的 PID 匿名身份机制 系统在用户注册时由平台服务器基于主密钥 K_{master} 、用户真实标识 $userID$ 和随机数 r 通过 HMAC-SM3 生成匿名身份标识 PID，即 $PID = \text{HMAC}(K_{master}, userID \parallel r)$ 。PID 在订单通信过程中替代真实身份参与认证和通信，骑手端和管理员端默认不直接展示 PID 原文，而是展示其 SM3 摘要值。该机制遵循最小知晓原则，在保障业务功能的前提下最大限度减少身份信息的暴露。

(2) 基于 HMAC-SM3 的订单 Token 认证机制 用户创建订单后，服务器以订单号 $orderID$ 、匿名身份标识 PID 、时间戳 $timestamp$ 和 128 bit 随机数 $nonce$ 作为待认证消息，使用订单认证密钥 K_{order} 计算 HMAC-SM3 值作为订单 Token。用户进入订单通信通道时需提交订单号和 Token，服务器重新计算并验证 Token 的合法性、订单归属关系和时效性。时间戳和随机数的引入有效降低了 Token 被重放或预测的风险。

(3) 基于 SM4-CBC 的消息加密存储与 SM3 哈希审计 用户与骑手之间的通信消息经服务器处理后，使用 SM4-CBC 模式进行加密存储，管理员端默认不展示消息明文，而是展示消息哈希、时间戳、订单号等审计信息。同时，系统对每条消息日志计算绑定订单号、消息序号、发送方角色、时间戳、密文摘要和前序哈希的 SM3 摘要作为 Merkle Tree 叶子节点，为日志完整性验证奠定基础。

(4) 基于 Merkle Tree 的日志完整性验证机制 系统采用快照机制保存订单通信日志的 Merkle Root。当订单通信结束或管理员手动触发快照时，系统对当前订单的消息日志集合计算 Merkle Root 并写入快照表。后续验证时重新计算 Root 并与历史快照对比，若不一致则说明日志被篡改。在理想哈希假设下，攻击者伪造指定 SM3 摘要的复杂度约为 2^{256} ，寻找任意碰撞的复杂度约为 2^{128} ，为日志完整性提供了较高强度的密码学保障。

(5) 条件溯源与可信审计闭环 当发生投诉或审计需求时，用户或骑手可基于订单号提交溯源申请，管理员经权限校验后执行条件溯源流程。溯源过程以订单号为入口，经 PID 映射恢复用户真实身份，且全程记录审计日志。溯源前必须先通过 Merkle Tree 完整性验证，确保溯源所依据的通信记录未被篡改。

系统的关键技术参数如表2所示。

表 2: FoodShield 系统关键技术参数

技术组件	参数	说明
SM3 密码杂凑	256 bit 输出	用于消息摘要、PID 哈希、Merkle 节点计算
SM4 对称加密	128 bit 密钥 / 128 bit 分组	CBC 模式，用于消息加密存储
HMAC-SM3	256 bit 输出	用于 PID 生成和订单 Token 计算
订单 Token	含 128 bit nonce	防重放设计，绑定订单号与匿名身份
Merkle Root	256 bit	日志完整性标识，通过快照机制保存
PID	HMAC 输出	匿名身份标识，与真实身份受控映射

1.5 应用前景

FoodShield 系统的设计理念和技术方案具有广阔的应用前景，主要体现在以下三个方面：

(1) 外卖行业隐私保护与合规管理 随着《个人信息保护法》和《数据安全法》的深入实施，外卖平台面临日益严格的隐私保护合规要求。FoodShield 提供的身份匿名、消息加密、日志完整性验证和条件溯源机制，可直接应用于外卖平台的订单通信系统，帮助平台在满足“最小必要原则”的前提下实现隐私保护与平台管理的平衡。系统中的虚拟号码脱敏、订单通信加密和审计日志等技术，为平台应对监管检查和用户投诉提供了技术支撑。

(2) 即时配送场景的安全通信扩展 FoodShield 的可控匿名通信模型不仅适用于外卖场景，也可推广至快递配送、跑腿代购、即时零售等更广泛的即时配送场景。这些场景具有与外卖类似的通信需求和安全风险——用户与配送员之间需要实时沟通，但用户不希望暴露真实身份。系统的模块化设计使得核心安全机制（PID 生成、Token 认证、Merkle 审计等）可作为独立组件集成到不同平台的业务流程中。

(3) 平台纠纷处理与数字证据可信化 在涉及配送纠纷、恶意骚扰或法律诉讼的场景中，通信记录的完整性和真实性是关键证据。FoodShield 的 Merkle Tree 日志完整性验证机制为通信记录提供了密码学完整性证明，使通信记录不仅能够被保存，而且能够被独立验证。这一机制可推广应用于电商平台客服聊天、在线医疗问诊、共享经济平台沟通等需要可信通信记录的场景，为数字时代的电子证据可信化提供技术方案。

第二章 预备知识

本章围绕 FoodShield 系统设计中所涉及的核心技术进行介绍, 主要包括 HMAC 消息认证码、SM3 密码杂凑算法、SM4 对称加密算法、PID 匿名身份机制、WebSocket 实时通信机制、Merkle Tree 完整性验证机制以及条件溯源与可控匿名模型。上述技术共同支撑了本作品在外卖订单通信场景下的身份匿名、订单认证、消息保护、日志审计和异常追责等安全目标。其中, SM3 和 SM4 算法均为我国商用密码标准算法, 分别遵循 GB/T 32905-2016 和 GB/T 32907-2016 国家标准; HMAC 机制遵循 RFC 2104 规范; WebSocket 协议遵循 RFC 6455 规范。

2.1 HMAC 消息认证码

HMAC (Hash-based Message Authentication Code) 是一种基于密码杂凑函数和共享密钥构造的消息认证码机制, 由 Bellare、Canetti 和 Krawczyk 于 1996 年提出, 并于 1997 年标准化为 RFC 2104[?]. 与单纯的哈希运算不同, HMAC 在计算过程中引入了密钥, 因此不仅能够检测消息内容是否被篡改, 还能够验证消息是否由持有合法密钥的一方生成。HMAC 通常用于身份认证、接口鉴权、请求防伪造和数据完整性校验等场景。

设 H 表示密码杂凑函数, K 表示共享密钥, m 表示待认证消息, 则 HMAC 的一般形式可以表示为:

$$\text{HMAC}(K, m) = H((K' \oplus \text{opad}) \parallel H((K' \oplus \text{ipad}) \parallel m)) \quad (1)$$

其中, K' 表示经过填充或压缩处理后的密钥 (若密钥长度小于杂凑函数的分组长度, 则在右侧填充零; 若大于分组长度, 则先对密钥计算一次杂凑值), opad (outer pad) 和 ipad (inner pad) 分别表示外部填充值和内部填充值, $\parallel$ 表示字符串连接操作, $\oplus$ 表示按位异或。对于输出 256 bit 摘要的杂凑函数 (如 SM3), ipad 为字节 0x36 重复 32 次构成的 512 bit 序列, opad 为字节 0x5C 重复 32 次构成的 512 bit 序列。

HMAC 的安全性取决于底层杂凑函数的性质和密钥的保密性。在形式化安全分析中, HMAC 的安全性通常以**选择消息攻击下的存在性不可伪造性** (Existential Unforgeability under Chosen-Message Attack, EUF-CMA) 来刻画: 对于概率多项式时间的攻击者 $\mathcal{A}$, 若其不知道密钥 K , 但在可以查询任意消息 m_i 对应的 $\text{HMAC}(K, m_i)$ 的条件下, 仍然难以构造出一个未查询过的消息 m^* 的合法认证码 $\text{HMAC}(K, m^*)$, 则称该 HMAC 方案满足 EUF-CMA 安全性。当底层杂凑函数满足抗碰撞性和伪随

机函数性质时，HMAC 的安全性可以归约到杂凑函数的安全性质上。

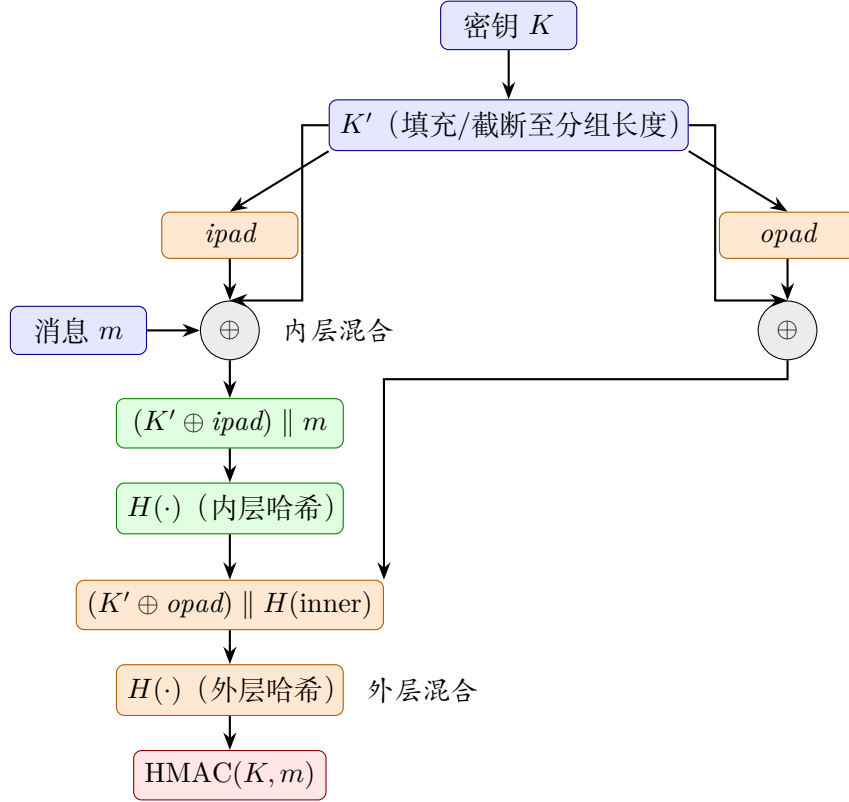


图 1: HMAC 双层嵌套哈希结构

在 FoodShield 系统中，HMAC 主要用于订单 Token 的生成与验证。用户在创建订单后，平台服务器根据订单号、匿名身份标识、时间戳和随机数等字段生成订单认证 Token。用户进入订单通信通道时需要提交订单号和 Token，服务器重新计算并比对 Token，以判断用户是否为该订单的合法参与方。该机制使用户能够在不直接暴露真实身份的情况下证明自己拥有订单通信资格。

本系统中订单 Token 的抽象形式如下：

$$m_{\text{order}} = \text{orderID} \parallel \text{PID} \parallel \text{timestamp} \parallel \text{nonce} \quad (2)$$

$$\text{token} = \text{HMAC}(K_{\text{order}}, m_{\text{order}}) \quad (3)$$

其中， K_{order} 为订单认证密钥， orderID 为订单编号， PID 为用户匿名身份标识， timestamp 为时间戳， nonce 为 128 bit 随机数。通过引入时间戳和随机数，可以降低 Token 被重放利用的风险——即使攻击者截获了某次通信的 Token，由于时间戳的时效性约束和随机数的不可预测性，该 Token 也无法在其他订单或其他时间窗口

中被成功复用。通过绑定订单号与匿名身份标识，可以防止攻击者将一个订单的认证凭证迁移到其他订单中使用。

2.2 SM3 密码杂凑算法

SM3 是我国商用密码标准中的密码杂凑算法，于 2016 年作为国家标准 GB/T 32905-2016 发布 [?]。SM3 的输出长度固定为 256 bit，分组长度为 512 bit，采用 Merkle-Damgård 迭代结构，共执行 64 轮压缩运算。SM3 主要用于数据完整性校验、数字签名、消息认证码构造和随机数生成等场景，已成为我国金融、政务、电力等关键信息基础设施中广泛部署的国密算法之一。

设 m 表示输入消息，SM3 的输出可以表示为：

$$h = \text{SM3}(m) \quad (4)$$

其中， h 为长度为 256 bit 的消息摘要。SM3 的压缩函数将 256 bit 的中间状态值和 512 bit 的消息分组作为输入，通过 64 轮混合运算（包括消息扩展、布尔函数、模加运算和循环移位等操作）输出新的 256 bit 状态值。由于摘要长度固定，SM3 适合用于对任意长度数据生成紧凑的完整性标识。

密码杂凑函数的安全性通常以以下三个性质来刻画：

- **抗原像性 (Preimage Resistance)**：给定哈希值 h ，难以找到任意消息 m 使得 $\text{SM3}(m) = h$ 。
- **抗第二原像性 (Second Preimage Resistance)**：给定消息 m_1 ，难以找到另一个不同的消息 $m_2 \neq m_1$ 使得 $\text{SM3}(m_1) = \text{SM3}(m_2)$ 。
- **抗碰撞性 (Collision Resistance)**：难以找到任意两个不同的消息 $m_1 \neq m_2$ 使得 $\text{SM3}(m_1) = \text{SM3}(m_2)$ 。

上述三个性质为 FoodShield 系统的安全机制提供了理论基础：抗原像性保证攻击者无法从消息摘要逆推原始消息内容；抗第二原像性保证攻击者无法为已有消息构造具有相同摘要的替代消息；抗碰撞性则是 Merkle Tree 完整性验证的安全基础——只要攻击者无法找到 SM3 碰撞，就无法在不改变 Merkle Root 的前提下篡改任意叶子节点 [?, ?]。

在 FoodShield 系统中，SM3 主要用于三个方面。第一，SM3 可作为 HMAC 的底层杂凑函数，用于构造 HMAC-SM3 订单认证 Token。第二，系统可以对通信消

息、密文内容、订单编号、发送方角色、时间戳等字段计算消息摘要，作为日志完整性验证的基础。第三，系统可以对 PID、设备标识等敏感字段进行摘要化处理，使管理员端在默认情况下只看到摘要值，而不直接接触敏感原始信息。

例如，对一条订单通信消息，可以将其摘要定义为：

$$message_{hash} = SM3(orderID \parallel senderRole \parallel timestamp \parallel ciphertext) \quad (5)$$

其中，*senderRole* 表示发送方角色，*ciphertext* 表示加密后的消息内容。只要消息内容、订单号、发送方角色或时间戳发生变化，重新计算得到的摘要值就会发生显著变化（雪崩效应）。因此，消息摘要可以作为后续 Merkle Tree 日志完整性验证的基本单元。

2.3 SM4 对称加密算法

SM4 是我国商用密码标准中的分组对称加密算法，于 2016 年作为国家标准 GB/T 32907-2016 发布 [?]。SM4 的分组长度为 128 bit，密钥长度为 128 bit，采用 32 轮非平衡 Feistel 结构，每轮使用相同的轮函数但以不同的轮密钥进行运算。SM4 的加密和解密使用相同的算法结构，区别仅在于轮密钥的使用顺序相反。对称加密算法的特点是加密和解密使用相同密钥，具有计算效率高、适合批量数据加密等优点，常用于通信数据保护、数据库字段加密和文件加密等场景。

设 K 表示对称加密密钥， M 表示明文消息， C 表示密文消息，则 SM4 加密与解密过程可以抽象表示为：

$$C = \text{Enc}_K(M) \quad (6)$$

$$M = \text{Dec}_K(C) \quad (7)$$

SM4 的轮函数由以下组件构成：输入分组的 4 个 32 bit 字中，第一个字与其余三个字经轮函数 F 处理后异或，再与轮密钥结合，生成下一轮的输入。轮函数 F 中包含一个 8 bit 输入/8 bit 输出的 S 盒（非线性置换）和一个线性变换 L （由循环移位和异或组成）。密钥扩展算法使用与加密算法类似的结构，将 128 bit 主密钥扩展为 32 个 32 bit 轮密钥。

在实际系统中，SM4 通常需要结合工作模式使用，以处理长度大于一个分组的数据并增强安全性。常见的工作模式包括：

- **ECB (Electronic Codebook) 模式**：每个分组独立加密，相同明文分组产生

相同密文，不适合加密结构化数据。

- **CBC (Cipher Block Chaining) 模式**：每个明文分组在加密前先与前一个密文分组异或，引入初始化向量 IV 使相同明文在不同加密过程中得到不同密文，安全性优于 ECB 模式。
- **CTR (Counter) 模式**：将递增计数器作为加密输入，输出与明文异或得到密文，支持并行加密。
- **GCM (Galois/Counter Mode) 模式**：在 CTR 模式基础上增加认证标签，同时提供加密和完整性保护。

本作品采用 CBC 模式，加密时需要引入初始化向量 IV ，使相同明文在不同加密过程中得到不同密文，从而降低明文模式泄露风险。其抽象形式如下：

$$C = \text{SM4-CBC-Enc}_K(IV, M) \quad (8)$$

具体而言，CBC 模式的加密过程为： $C_0 = IV$ ， $C_i = \text{Enc}_K(M_i \oplus C_{i-1})$ ，其中 M_i 和 C_i 分别表示第 i 个明文分组和密文分组。解密过程为： $M_i = \text{Dec}_K(C_i) \oplus C_{i-1}$ 。由于每个分组的加密都依赖于前一个密文分组，攻击者无法仅通过替换某个密文分组来篡改对应的明文而不影响后续分组的解密结果。

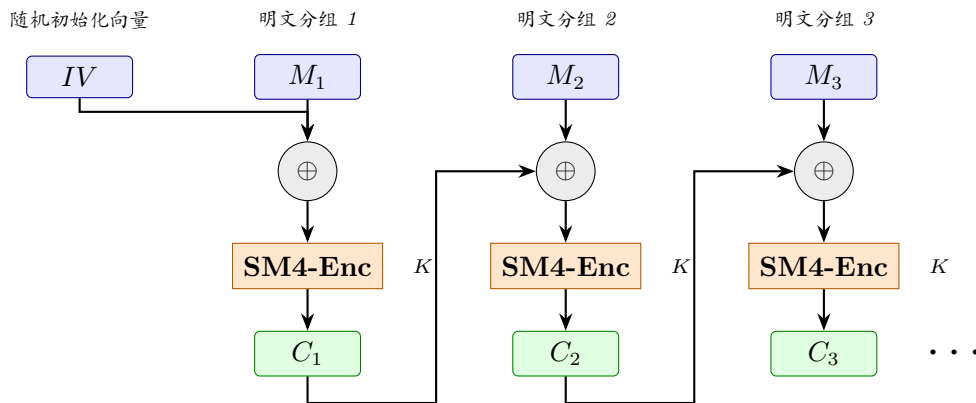


图 2: SM4-CBC 工作模式加密链示意图

在 FoodShield 系统中，SM4 主要用于订单通信消息的加密存储。用户与骑手之间的消息经过服务器处理后，使用订单级会话密钥或平台侧加密密钥进行加密，再写入数据库。管理员端默认不直接展示消息明文，而是展示消息哈希、时间戳、订单号和 Merkle Root 等审计信息。这样可以降低数据库泄露或管理员越权查看造成的隐私风险。

需要说明的是，本作品原型系统以可运行、可展示和安全机制闭环为主要目标。在实际部署环境中，通信链路还应结合 HTTPS/TLS 保障传输安全，SM4 更多用于服务端存储阶段的消息内容保护。通过“传输层加密 + 存储层加密 + 日志哈希审计”的组合设计，可以从多个层面降低订单通信数据泄露和篡改风险。

2.4 PID 匿名身份机制

PID (Pseudonymous Identifier) 即伪身份标识或匿名身份标识，是隐私保护系统中常用的一种身份匿名机制。其核心思想是：系统内部不直接使用真实用户身份参与通信和认证，而是为用户生成一个与真实身份存在受控映射关系的匿名标识。在日常业务过程中，通信参与方只能看到订单相关信息或匿名身份信息，不能直接获得用户真实身份。

在 FoodShield 系统中，用户注册后由平台服务器生成 PID。PID 由主密钥、用户真实标识、随机数和设备摘要等字段通过 HMAC 机制生成，其抽象形式如下：

$$PID = HMAC(K_{\text{master}}, userID \parallel r) \quad (9)$$

其中， K_{master} 为平台主密钥， $userID$ 为用户真实身份标识， r 为随机数。引入随机数 r 的作用在于防止敌手实现**并发攻击** (Cross-Order Unlinkability)：即使同一用户在不同订单中使用 PID，由于每次生成的随机数不同，攻击者无法通过比较 PID 值来判断两个订单是否属于同一用户。这一性质对于防止跨订单的用户画像构建和身份关联攻击具有重要意义。

PID 的安全性可以从以下两个角度分析：

- **不可逆性**：由于 PID 基于 HMAC-SM3 生成，而 HMAC 的输出在不知道密钥 K_{master} 的情况下无法逆推出输入 $userID \parallel r$ ，因此攻击者即使获取了 PID，也无法直接恢复用户的真实身份。该性质基于 SM3 的抗原像性和 HMAC 的 EUF-CMA 安全性。
- **不可链接性**：由于每次 PID 生成引入了不同的随机数 r ，即使攻击者获取了同一用户的多个 PID，也无法判断这些 PID 是否指向同一用户。该性质基于 HMAC 输出的伪随机性。

PID 的作用不是取代用户登录身份，而是在订单通信过程中隐藏真实身份。用户在系统中仍然需要通过用户名、口令和设备识别等方式完成正常登录；登录成功

后，系统在内部使用 PID 与订单进行绑定。骑手端只需要知道订单编号、订单状态和必要配送信息，不应看到用户真实身份，也不应直接看到 PID。管理员端默认也不直接展示 PID，而是展示经过摘要化处理的 PID 哈希值。

因此，FoodShield 中 PID 的设计原则可以概括为三点。第一，PID 是系统内部匿名标识，不作为普通前端页面中的公开身份展示。第二，PID 与真实身份之间的映射关系只能由平台服务器在受控条件下查询。第三，当发生投诉、恶意骚扰、纠纷处理或审计需求时，系统可以通过条件溯源流程恢复必要身份信息，从而实现隐私保护与责任追踪之间的平衡。

2.5 WebSocket 实时通信机制

WebSocket 是一种基于 TCP 的全双工通信协议，由 Fette 和 Melnikov 于 2011 年标准化为 RFC 6455[?]。与传统的 HTTP 通信采用“客户端请求、服务器响应”的模式不同，WebSocket 在客户端与服务器建立连接后，可以在同一连接上进行双向、全双工的实时数据交互，因此常用于在线聊天、实时通知、协同编辑和数据监控等场景。

WebSocket 通信的一般过程包括连接建立、身份认证、消息发送、消息转发和连接关闭五个阶段。在连接建立阶段，客户端首先向服务器发送 HTTP 升级请求 (Upgrade: websocket)，服务器返回 101 Switching Protocols 响应后双方建立 WebSocket 长连接。此后，客户端和服务器可以在该连接上持续交换数据帧 (Data Frame)，而不需要为每条消息重新建立 HTTP 请求。这一机制显著降低了通信延迟和带宽开销，适合需要频繁消息交互的实时场景。

在 FoodShield 系统中，WebSocket 用于实现用户端与骑手端之间的订单级实时通信。系统以订单号为基本单位建立通信房间，每个订单对应一个独立的通信通道。用户或骑手进入通信房间前，服务器需要验证其身份状态和订单参与资格。对于用户端，服务器需要校验订单号和 Token；对于骑手端，服务器需要校验其是否为该订单绑定的配送方。只有验证通过的参与方才能加入对应订单房间。

订单通信过程可以抽象为：

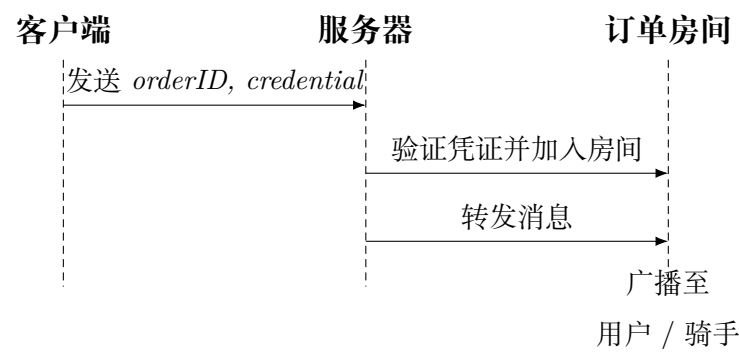


图 3: 订单认证与消息转发时序图

其中，*credential* 表示订单认证凭证（即 Token）或骑手身份凭证。服务器在转发消息的同时，对消息进行加密存储、哈希计算和日志记录，使通信功能与安全审计机制形成联动。

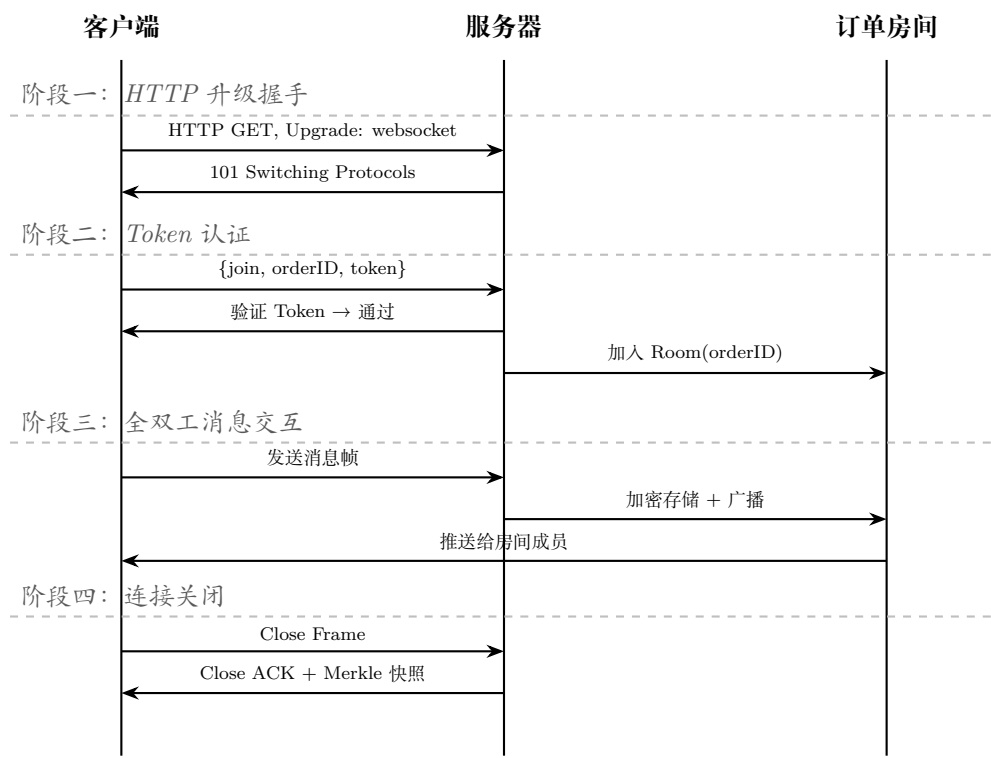


图 4: WebSocket 握手及订单通信时序示意图

需要指出的是，WebSocket 协议本身主要解决实时通信的连接管理问题，并不直接提供消息加密、身份认证或日志完整性保护等安全功能。因此，在实际应用中，WebSocket 应结合 TLS 形成 WSS（WebSocket Secure）安全连接，以防止通信内容在传输过程中被窃听或篡改。本作品在系统设计中将 WebSocket 作为实时消息通道，将 Token 认证、消息加密、哈希审计和权限控制作为安全增强机制，从而避免将普通聊天功能误认为完整安全机制。

2.6 Merkle Tree 完整性验证

Merkle Tree 是一种基于哈希函数构造的树形数据完整性验证结构，又称哈希树，由 Ralph Merkle 于 1979 年提出 [?]。其基本思想是：先对每个数据块计算哈希值，作为叶子节点；再将相邻节点哈希拼接后继续计算父节点哈希；最终逐层向上计算得到根节点哈希，即 Merkle Root。只要任意一个叶子节点对应的数据发生变化，最终计算得到的 Merkle Root 就会发生变化。因此，Merkle Tree 广泛应用于区块链、文件完整性验证、日志审计和分布式存储等场景。

设系统中共有 n 条消息日志，分别为 $m_1, m_2, \dots, m_n$ ，对应叶子节点可以表示为：

$$leaf_i = H(m_i), \quad i = 1, 2, \dots, n \quad (10)$$

相邻两个叶子节点进一步计算父节点：

$$parent_{i,j} = H(leaf_i \parallel leaf_j) \quad (11)$$

逐层向上递归计算，最终得到根节点：

$$root = \text{MerkleRoot}(leaf_1, leaf_2, \dots, leaf_n) \quad (12)$$

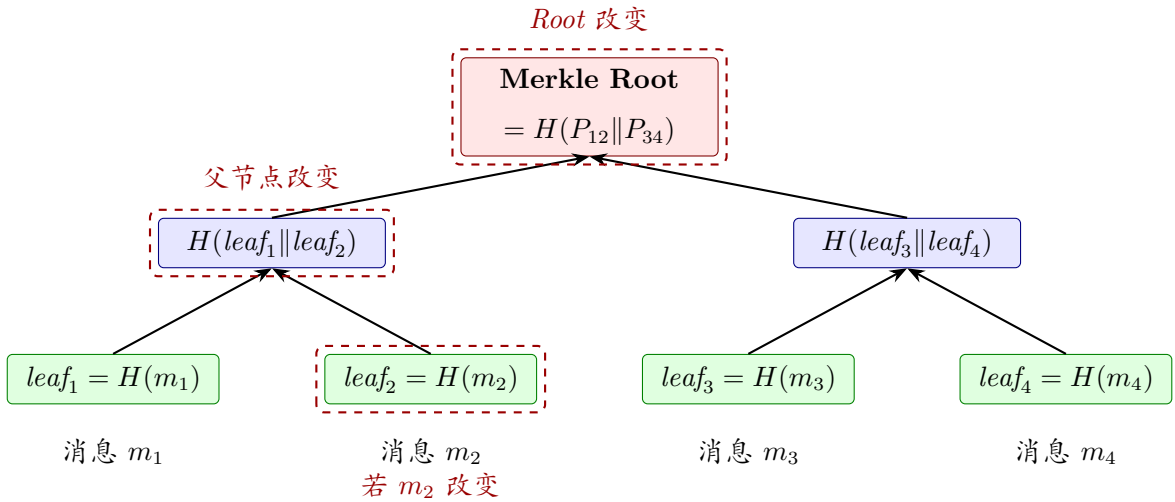


图 5: Merkle Tree 结构示意图（任意叶子变化均导致 Root 改变）

Merkle Tree 的一个重要优势是支持高效的**成员关系证明** (Inclusion Proof)：要验证某个叶子节点 $leaf_i$ 是否包含在以 $root$ 为根的 Merkle Tree 中，验证者只需知道

从 $leaf_i$ 到 $root$ 路径上的兄弟节点哈希值（即验证路径），即可通过 $O(\log n)$ 次哈希计算完成验证，而无需获取全部 n 个叶子节点。这一性质使得 Merkle Tree 在大规模数据集的完整性验证中具有显著的效率优势。

在 FoodShield 系统中，Merkle Tree 用于订单通信日志的完整性验证。系统并不直接将明文消息作为叶子节点，而是将与消息相关的摘要信息作为叶子节点输入。为了绑定订单上下文、发送方角色、消息顺序和密文内容，本系统中单条消息日志的叶子节点可以抽象定义为：

$$leaf_i = \text{SM3}(\text{orderId} \parallel \text{msgSeq} \parallel \text{senderRole} \parallel \text{timestamp} \parallel \text{ciphertextHash} \parallel \text{prevHash}) \quad (13)$$

其中，各字段含义如下：

- *orderId*: 订单编号，将叶子节点绑定到特定订单，防止跨订单拼接攻击。
- *msgSeq*: 消息序号，记录消息在订单通信中的顺序，可检测消息重排和删除。
- *senderRole*: 发送方角色（用户或骑手），防止发送方身份伪造。
- *timestamp*: 发送时间戳，提供时间维度的可验证性。
- *ciphertextHash*: 消息密文摘要，绑定消息内容，检测内容篡改。
- *prevHash*: 前一条消息日志的摘要，形成链式结构，可检测消息插入和删除。

该设计通过将多个上下文字段绑定到叶子节点中，能够同时检测消息内容篡改、消息顺序调整、消息插入和消息删除等异常情况。与仅对密文计算哈希的简单方案相比，本方案的叶子节点结构提供了更细粒度的完整性保护。

为了保证 Merkle Root 的可验证性，系统采用快照机制保存 Root。每当订单通信结束、管理员手动生成审计快照或消息数量达到设定阈值时，系统对当前订单的消息日志集合计算 Merkle Root，并将订单号、消息数量、Root 值和快照时间写入日志快照表。后续进行完整性验证时，系统重新读取订单消息日志并计算新的 Root，与历史快照中的 Root 进行比较。如果二者一致，说明当前日志集合与快照生成时保持一致；如果二者不一致，则说明日志可能被修改、删除、插入或重排。

因此，Merkle Tree 在本系统中的作用不是加密消息内容，而是提供日志完整性证明。它可以使平台在发生纠纷时证明通信记录是否被篡改，从而提升订单通信记录的可信度和审计价值。

2.7 条件溯源与可控匿名模型

匿名机制能够保护用户隐私，但完全匿名也可能导致恶意行为难以追责。在外卖订单通信场景中，用户和骑手之间可能发生恶意骚扰、虚假投诉、配送纠纷或违规沟通等情况。如果系统只强调匿名而不保留管理能力，平台将难以处理异常事件；如果系统默认暴露真实身份，则又会削弱隐私保护效果。因此，本作品采用条件溯源与可控匿名相结合的设计思路。

可控匿名（Controllable Anonymity）是指系统在正常业务流程中隐藏用户真实身份，而在满足特定条件时，由具备权限的管理方按照审计流程恢复必要身份信息。该模型强调“日常匿名、异常可追责”。可控匿名区别于完全匿名（如 Tor）和完全实名两种极端模式，在隐私保护与责任追踪之间寻求平衡。

在 FoodShield 系统中，普通通信状态下，骑手端无法看到用户真实身份，也无法看到用户 PID；管理员端默认只查看订单摘要、消息哈希、Merkle Root 和完整性验证结果。当发生投诉、纠纷或平台审计需求时，用户或骑手可以基于订单号提交溯源申请，管理员经过身份认证和权限校验后，才能触发条件溯源流程。

条件溯源流程可以抽象为：

$$orderID \xrightarrow{\text{verify-integrity}} PID \xrightarrow{\text{lookup}} userID \quad (14)$$

其中， $orderID$ 是业务入口， PID 是系统内部匿名标识， $userID$ 是真实用户身份标识。系统不鼓励管理员直接通过 PID 发起溯源，而是以订单号作为溯源入口。这种方式更符合外卖平台的真实业务逻辑（投诉和纠纷通常以订单为单位发起），也可以降低 PID 被滥用或过度暴露的风险。

条件溯源流程包含两个关键安全约束：

- **完整性前置校验**：在执行溯源之前，系统必须先对目标订单的通信日志进行 Merkle Tree 完整性验证。只有当日志完整性验证通过时，才允许继续溯源流程。这一约束确保了溯源所依据的通信记录未被篡改，防止攻击者通过篡改日志来误导溯源结果。
- **关键词条件触发**：溯源过程通常由投诉或举报触发，系统根据投诉内容中的关键词进行命中匹配，确认是否存在异常通信行为。只有当关键词命中且完整性校验通过时，才执行从 $orderID$ 到 $userID$ 的身份映射。这一双重门槛机制防止管理员滥用溯源权限进行无依据的身份查询。

在条件溯源过程中，系统应记录管理员的关键操作，包括登录行为、查看订单摘要、执行完整性验证、确认溯源申请和查询身份映射等。每一次溯源操作都应写入审计日志，记录操作者、操作时间、目标订单、溯源理由和操作结果。通过这种方式，系统不仅可以追踪用户或骑手的异常行为，也可以约束管理员权限，避免审计能力本身成为新的隐私风险。

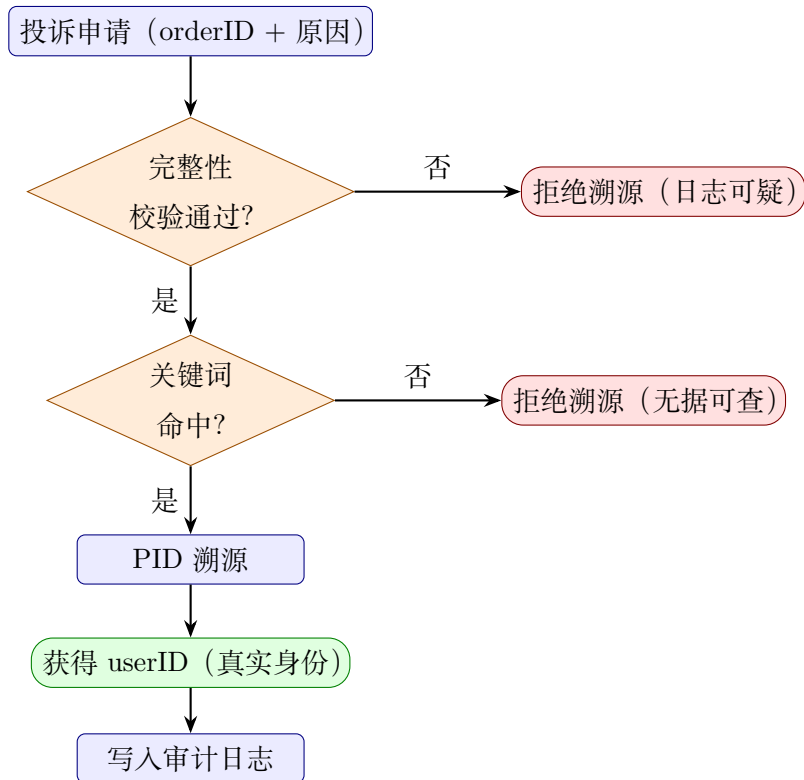


图 6: 条件溯源概念流程（预备知识层）

综上，条件溯源与可控匿名模型是 FoodShield 系统区别于普通匿名通信系统的重要设计。它既避免了普通外卖聊天系统中过度暴露身份的问题，也避免了完全匿名系统中责任主体无法追踪的问题，从而在隐私保护、订单认证和平台管理之间形成平衡。

第三章 系统设计

3.1 系统总体方案

FoodShield 是一个面向外卖订单通信场景的隐私认证与可信审计系统。系统以外卖平台中用户与骑手之间的订单沟通为基本应用场景，围绕身份信息暴露、订单隐私泄露、聊天记录篡改和纠纷追责困难等问题，设计了集匿名身份、订单认证、加密通信、日志完整性验证和条件溯源于一体的安全通信方案。

传统外卖订单通信系统通常更关注业务连通性，即用户和骑手能否完成消息交互，而对通信过程中的安全边界关注不足。例如，骑手端可能接触到超出配送所需范围的用户信息，普通订单号可能被误用或伪造，管理员端可能拥有过大的信息查看权限，历史通信记录也可能在纠纷处理前被修改、删除或重排。针对上述问题，FoodShield 将“订单”作为系统安全控制的核心索引，将用户身份、订单资格、通信通道和审计日志统一绑定到订单生命周期中，从而在业务可用性的基础上增加隐私保护和可信审计能力。

系统总体设计遵循“日常匿名、订单认证、日志可信、异常可溯源”的原则。在日常通信过程中，系统不向骑手端暴露用户真实身份，也不向骑手端展示用户 PID；用户注册阶段不再是简单输入用户名生成匿名标识，而是采用“用户名、口令、机器识别”相结合的方式，降低机器批量注册和匿名身份滥用风险。用户创建订单后，系统将订单号、用户匿名身份、骑手身份和订单状态进行绑定，并生成基于 HMAC-SM3 的订单 Token。用户进入订单通信通道前，需要提交订单号和 Token，服务器验证通过后才允许其加入对应订单房间。

通信过程中，用户端与骑手端通过订单级 WebSocket 通道进行实时通信。系统对通信消息进行加密存储，并对订单号、发送方角色、时间戳、消息序号和密文摘要等字段计算消息哈希。审计阶段，系统将消息摘要作为 Merkle Tree 叶子节点，生成 Merkle Root 并通过快照机制保存。后续若通信记录被修改、删除、插入或重排，重新计算得到的 Root 将与历史快照不一致，从而能够检测日志篡改行为。

当发生投诉、恶意骚扰、虚假争议或平台审计需求时，系统支持用户端和骑手端基于订单号发起条件溯源申请。管理员登录后只能默认查看订单摘要、消息哈希、Merkle Root 和审计记录，不直接展示通信明文内容；只有在满足权限校验、审计理由和操作记录要求后，管理员才能基于订单号触发条件溯源。该设计避免了直接通过 PID 任意查询真实身份的问题，使系统在保护用户隐私的同时保留必要的平台管理能力。

从功能组成上看，FoodShield 主要包括以下核心模块：

1. **用户注册与身份初始化模块**：负责用户注册、口令认证、机器识别、PID 生成和匿名身份初始化，避免系统仅具有标识而缺少真实认证流程。
2. **匿名身份管理模块**：负责生成和维护用户 PID，并保存 PID 与真实身份之间的受控映射关系。PID 默认不在普通前端页面展示，骑手端不可见。
3. **订单 Token 认证模块**：负责根据订单号、PID、时间戳和随机数生成订单认证 Token，并在用户进入通信通道前进行合法性、归属关系和时效性验证。
4. **订单级实时通信模块**：负责用户端与骑手端之间的 WebSocket 通信、订单房间管理、消息转发、订单历史记录和订单搜索。
5. **消息保护与日志审计模块**：负责消息加密存储、SM3 摘要生成、Merkle Tree 构建、Merkle Root 快照保存和日志完整性验证。
6. **条件溯源与管理审计模块**：负责投诉申请、管理员权限校验、基于订单号的条件溯源和管理员操作审计记录。

在技术实现上，系统采用 Python Flask 作为后端服务框架，使用 SQLite 存储用户、订单、消息和审计数据，使用 WebSocket 实现实时通信。在密码学机制方面，系统使用 HMAC-SM3 构造订单认证 Token，使用 SM4 对通信消息进行加密存储，使用 SM3 生成消息摘要，并使用 Merkle Tree 对通信日志进行完整性验证。上述机制共同构成了 FoodShield 的安全基础，使其能够在外卖订单通信这一具体场景中兼顾隐私保护、业务可用性和平台管理能力。

3.2 系统参与方与信任边界

FoodShield 系统涉及用户端、骑手端、平台服务器、管理员端和审计日志系统五类主要参与方。不同参与方具有不同的业务权限和信息可见范围。系统通过权限隔离、字段隐藏和数据最小化原则限制敏感信息暴露，避免普通通信功能演变为身份泄露通道。

表 3: 系统主要参与方与权限边界

参与方	主要功能	权限与隐私边界
用户端	注册登录、创建订单、查看订单历史、订单搜索、添加备注标签、发起投诉申请。	仅可查看本人订单与本人通信内容、PID 作为系统内部匿名标识，默认不在普通页面展示、用户不直接接触真实身份与 PID 的映射关系。
骑手端	查看分配订单、搜索配送任务、进入订单通信、处理配送异常、发起投诉申请。	仅可查看订单编号、订单状态、必要配送信息和通信窗口、不可见用户真实身份、不可见用户 PID、不可见用户备注和标签原文。
平台服务器	用户认证、机器识别、PID 生成、订单绑定、Token 验证、消息转发、加密存储、条件溯源。	作为核心安全控制单元，维护必要的身份映射和订单状态、不向普通业务端暴露真实身份映射、敏感操作需要权限控制与审计记录。
管理员端	管理员登录、订单检索、日志验证、处理溯源申请、查看审计结果。	默认只查看订单摘要、消息哈希、Merkle Root、日志验证结果和溯源申请状态、默认不展示通信明文、用户备注和标签原文、不能直接按 PID 任意溯源。
审计日志系统	保存消息摘要、保存 Merkle Root 快照、记录管理员操作、支持日志完整性验证。	不作为明文消息查询入口、主要用于检测日志篡改，并为纠纷处理和管理员行为追踪提供可信依据。

从信任关系上看，FoodShield 采用平台可控匿名模型。平台服务器被视为系统中的基础可信实体，负责维护真实身份与 PID 之间的映射关系；用户和骑手彼此之间不完全信任，系统需要防止双方获取超过订单通信所需范围的敏感信息；管理员具有较高权限，但管理员行为也不能完全无约束，因此系统需要记录管理员的登录、

查询、验证和溯源操作；外部攻击者不被信任，可能尝试伪造订单身份、重放 Token、篡改通信日志或越权访问管理员接口。

系统的主要信任边界包括以下几个方面：

1. **用户端与骑手端之间的边界**：用户与骑手只应围绕订单配送进行通信。骑手端不应获得用户真实身份和 PID，用户端也不应获得骑手端超出配送所需范围的敏感信息。
2. **普通业务端与管理端之间的边界**：用户端和骑手端不能访问管理员接口；管理端访问审计功能前必须完成身份认证。
3. **平台服务器与数据库之间的边界**：数据库保存订单、消息和审计数据，但通信明文不应直接裸露存储。消息内容应加密保存，消息摘要参与完整性验证。
4. **管理员权限与溯源能力之间的边界**：管理员不能任意根据 PID 发起溯源，而应基于订单号、投诉理由和审计需求触发条件溯源，并将操作写入审计日志。
5. **备注标签与审计数据之间的边界**：用户可为订单添加备注或标签以便历史订单管理和搜索，但管理端默认不展示备注或标签的具体内容，避免审计权限扩大为隐私查看权限。
6. **日志数据与业务数据之间的边界**：日志审计系统主要保存消息摘要、Merkle Root 和操作记录，用于验证数据是否被篡改，而不是作为普通业务查询明文消息的入口。

通过上述信任边界划分，FoodShield 将用户隐私保护、订单通信业务和平台管理能力进行分层隔离。不同参与方只能访问其业务所需的最小信息集合，从而降低身份暴露、越权访问、日志篡改和审计权限滥用风险。

3.3 威胁模型

FoodShield 的威胁模型围绕外卖订单通信场景中的身份伪造、隐私泄露、日志篡改、管理员越权和匿名身份滥用等问题展开。系统假设攻击者具有一定的网络访问能力和业务接口调用能力，可能通过伪造请求参数、重放认证凭证、篡改数据库记录、批量注册账号或越权访问接口等方式破坏系统安全性。

本系统重点考虑以下几类威胁：

1. **订单身份伪造攻击**：攻击者可能尝试构造虚假的订单号或 Token，以非订单参与方身份进入用户与骑手之间的订单通信通道。如果系统仅依赖订单号作为入口，则订单号一旦泄露，攻击者可能冒充合法用户或骑手参与通信。
2. **Token 重放攻击**：攻击者可能截获某次合法订单认证请求中的 Token，并在之后重复使用该 Token 尝试进入通信通道。如果 Token 不包含时间戳、随机数或有效期限限制，则存在被重复利用的风险。
3. **机器批量注册攻击**：攻击者可能使用脚本批量注册账号，生成大量匿名身份并发起垃圾订单或骚扰通信。如果注册阶段仅输入用户名并直接生成 PID，则系统实际上只有标识生成过程，而缺少有效认证和反滥用能力。
4. **用户身份暴露风险**：在普通外卖通信模式下，用户真实身份、联系方式、地址备注等信息可能被骑手端或管理员端过度接触。恶意骑手可能利用固定身份标识进行跨订单关联，甚至造成骚扰或隐私泄露。
5. **通信日志篡改攻击**：攻击者或内部恶意人员可能尝试修改、删除、插入或重排历史通信记录，以影响订单纠纷处理结果。如果系统仅保存普通数据库记录，而缺少完整性验证机制，则难以证明记录是否保持原始状态。
6. **管理员越权访问风险**：管理员具有较高审计权限，如果缺少权限控制和审计记录，管理员可能在无投诉、无异常或无授权的情况下查看敏感数据或执行溯源操作，造成二次隐私泄露。
7. **订单备注与标签泄露风险**：订单备注或标签可能包含用户生活习惯、地址细节、偏好信息等隐私内容。如果管理员端默认展示这些内容，可能导致审计功能扩大为隐私查看功能。

针对上述威胁，FoodShield 采用多层防护机制。对于订单身份伪造问题，系统使用 HMAC-SM3 生成订单 Token，将订单号、PID、时间戳和随机数绑定到同一认证凭证中；对于 Token 重放风险，系统引入时间戳和随机数，并设置 Token 有效期；对于机器注册攻击，系统在注册阶段引入口令认证和设备识别，必要时结合注册频率限制；对于身份暴露风险，系统通过 PID 匿名身份机制隐藏真实用户身份，并限制骑手端和管理员端的可见字段；对于通信日志篡改问题，系统使用 SM3 消息摘要和 Merkle Tree 完整性验证机制检测历史记录是否被修改；对于管理员越权问题，系统要求管理员登录认证，并将关键操作写入审计日志；对于备注标签泄露问题，管理员端默认不展示用户备注或标签原文。

需要说明的是，本系统不追求对平台服务器的绝对匿名。平台服务器作为业务和安全控制中心，需要维护用户真实身份与 PID 的映射关系，以便在投诉、纠纷或审计场景下进行条件溯源。因此，FoodShield 采用的是“平台可控匿名”模型，而不是完全匿名模型。该模型的目标是在日常通信中对骑手、普通用户和外部攻击者隐藏真实身份，同时在满足审计条件时保留必要的责任追踪能力。

3.4 安全目标

结合外卖订单通信场景和上述威胁模型，FoodShield 的安全目标主要包括匿名性、认证性、机密性、完整性、可追责性、可审计性和抗滥用能力。系统通过不同安全机制分别支撑上述目标，使外卖订单通信过程不仅能够完成业务沟通，而且能够满足基本的信息安全要求。

其中，匿名性和可追责性是本系统设计中的一组核心平衡目标。完全匿名虽然能够增强隐私保护，但可能导致恶意行为难以管理；完全实名虽然便于追责，但会增加用户身份暴露风险。因此，FoodShield 采用可控匿名设计：日常通信中隐藏真实身份，异常场景下通过管理员权限校验和审计记录完成条件溯源。

认证性是系统防止越权通信的关键。用户进入订单通信通道前，需要提交订单号和订单 Token，服务器根据订单绑定关系重新计算 Token 并判断其合法性。由于 Token 由订单号、PID、时间戳和随机数共同参与生成，攻击者在不知道订单认证密钥的情况下难以伪造合法凭证。

完整性是系统可信审计能力的基础。FoodShield 不仅保存通信记录，还对每条消息生成摘要，并将消息摘要组织为 Merkle Tree。系统通过保存 Merkle Root 快照，使后续审计过程中能够验证历史通信记录是否发生变化。如果任意消息内容、消息顺序或消息数量被修改，重新计算的 Root 将与历史快照不一致，从而暴露篡改行为。

可审计性主要用于约束管理员权限。管理员虽然可以处理异常订单和执行溯源，但其操作必须被记录。系统通过审计日志保存管理员身份、操作时间、目标订单、操作类型和操作结果，使管理员行为具备可追踪性，从而降低审计能力本身带来的隐私风险。

3.5 系统总体架构

FoodShield 采用多端协同的分层架构，整体上可以划分为表现层、业务服务层、安全机制层、数据存储层和审计验证层。表现层包括用户端、骑手端和管理员端；业

表 4: FoodShield 系统安全目标与对应机制

安全目标	防护对象	对应机制
匿名性	用户真实身份、用户 PID、跨订单关联信息	PID 内部化; 骑手端不可见 PID; 前端字段最小化展示
认证性	订单通信入口、订单参与资格	HMAC-SM3 订单 Token; 绑定 orderID、PID、timestamp、nonce
机密性	订单聊天内容、数据库消息记录	WebSocket/WSS 传输保护; SM4 消息加密存储; 管理员端默认不看明文
完整性	历史通信记录、消息顺序、消息数量	SM3 消息摘要; Merkle Tree; Merkle Root 快照比对
可追责性	投诉纠纷、恶意骚扰、异常订单	用户端与骑手端均可按 orderID 发起申请; 管理员条件溯源
可审计性	管理员查询、验证、溯源等高权限操作	管理员认证; audit_logs 记录操作时间、目标订单和操作结果
抗滥用能力	机器注册、垃圾订单、匿名身份滥用	用户名与口令认证; 机器识别; 注册频率限制
隐私最小化	用户备注、标签、通信明文、身份映射关系	管理员端仅展示订单摘要、消息哈希、Merkle Root 和验证状态

务服务层负责注册登录、创建订单、订单历史、订单搜索、备注标签、订单认证、消息转发、投诉申请等功能; 安全机制层提供 PID 生成、Token 验证、SM4 加密、SM3 摘要和 Merkle Tree 构建等能力; 数据存储层保存用户、骑手、订单、消息、快照和审计日志等数据; 审计验证层负责日志完整性验证、Root 快照比对、条件溯源记录和三端摘要一致性增强。

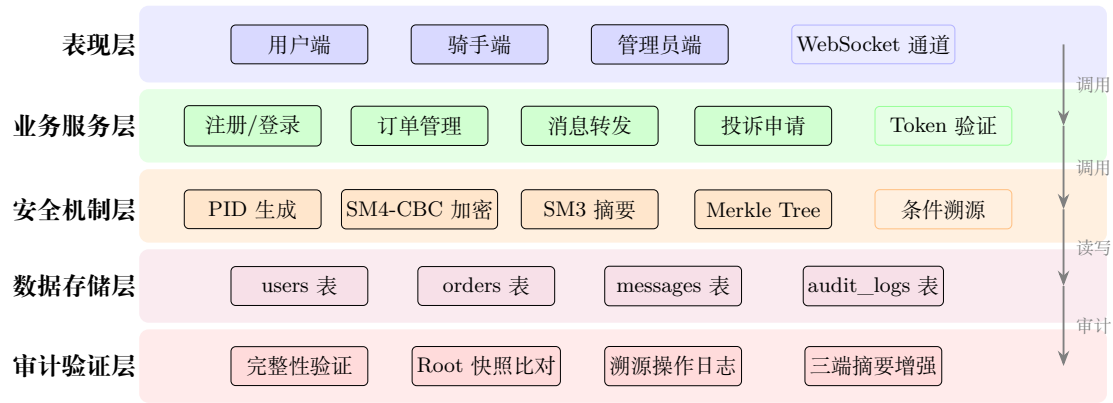


图 7: FoodShield 系统总体架构分层图

系统总体架构可以概括如下：

- 1. 用户端：**用户端提供用户注册、登录、创建订单、查看订单历史、搜索订单、添加备注标签、进入订单通信和提交投诉申请等功能。用户完成注册后，系统在内部生成 PID，并在订单创建时将 PID 与订单号进行绑定。用户进入通信通道前，需要提交订单号和 Token 完成订单认证。
- 2. 骑手端：**骑手端提供骑手登录、查看分配订单、根据订单号搜索订单、进入订单通信和提交异常申请等功能。骑手端只显示订单相关必要信息，不显示用户真实身份和 PID，避免骑手通过长期标识进行跨订单关联。
- 3. 平台服务器：**平台服务器是系统核心部分，负责接收用户端、骑手端和管理员端请求，并调用不同安全模块完成业务处理。平台服务器内部包括用户注册与机器识别模块、匿名身份管理模块、订单认证模块、消息转发模块、消息加密存储模块、日志审计模块和条件溯源模块。
- 4. 管理员端：**管理员端用于平台审计和异常处理。管理员登录后可以根据订单号检索订单摘要，查看消息哈希和 Merkle Root，执行日志完整性验证，处理用户端或骑手端提交的溯源申请。管理员端默认不展示通信明文内容，也不展示用户备注或标签原文。
- 5. 审计日志系统：**审计日志系统保存通信消息摘要、Merkle Root 快照和管理员操作日志。系统在订单通信过程中持续生成消息哈希，并在订单结束或管理员触发快照时计算 Merkle Root。后续验证时，系统重新计算 Root 并与快照值对比，以判断日志是否被篡改。

6. **三端摘要一致性增强机制**: 在订单通信结束时, 系统可将订单通信摘要或 Merkle Root 摘要分别留存在用户端、骑手端和平台端。后续审计时, 若攻击者试图篡改平台侧数据库记录, 则需要同时修改三端摘要才能伪造一致结果, 从而提高日志篡改成本。

系统的基本业务流程如下。首先, 用户完成注册和登录, 注册阶段包含用户名、口令和机器识别信息, 平台服务器生成用户 PID, 并保存 PID 与真实身份之间的受控映射关系。其次, 用户创建订单, 服务器生成订单号并将订单与用户 PID、骑手身份、订单备注和订单状态进行绑定, 同时生成订单 Token。随后, 用户进入订单通信页面时提交订单号和 Token, 服务器验证通过后允许用户加入对应 WebSocket 通信房间; 骑手端经过身份验证后进入同一订单房间。通信过程中, 平台服务器负责消息转发、消息加密存储和消息哈希计算。订单完成或管理员生成审计快照时, 系统根据消息摘要构建 Merkle Tree 并保存 Merkle Root。若后续发生投诉或纠纷, 用户或骑手可基于订单号发起申请, 管理员经权限验证后查看审计摘要、验证日志完整性, 并在必要时执行条件溯源。

在架构设计上, FoodShield 的关键特点在于将业务流程和安全机制进行绑定。订单不是单纯的业务编号, 而是认证、通信、日志和溯源的核心索引; PID 不是公开展示的用户身份, 而是内部匿名绑定标识; Token 不是普通随机字符串, 而是基于 HMAC-SM3 的订单认证凭证; 消息记录不是普通数据库文本, 而是加密存储并参与 Merkle Tree 完整性验证的审计对象; 管理员端不是明文查看入口, 而是哈希摘要、Root 验证和条件溯源的受控审计入口。通过这种设计, 系统能够围绕订单这一核心业务对象建立完整的隐私保护与可信审计闭环。

3.6 数据库设计

FoodShield 使用 SQLite 作为数据存储后端, 数据库共设计四张核心表: `users` (用户表)、`orders` (订单表)、`messages` (消息表) 和 `audit_logs` (审计日志表)。各表字段的设计均遵循“最小必要原则”, 避免将超出业务所需的敏感信息明文保存在数据库中。

用户表 (`users`)

用户表存储用户的注册信息与匿名身份标识, 字段设计如表 5 所示。

表 5: users 表字段说明

字段名	类型	说明	安全设计意义
id	INTEGER	自增主键	内部索引，不对外暴露
username	TEXT UNIQUE	用户名	用于登录认证
pid	TEXT UNIQUE	匿名身份标识 (PID)	隐藏真实 id，用于通信和溯源
pid_r	TEXT	PID 生成随机数 r	仅服务端保存，用于条件溯源时重算 PID
password_hash	TEXT	口令哈希值	SM3(salt password)，不存明文口令
salt	TEXT	随机盐值	防彩虹表攻击，每用户独立随机
created_at	TIMESTAMP	注册时间	用于审计和频率限制参考

订单表 (orders)

订单表存储订单生命周期数据，字段设计如表 6 所示。

表 6: orders 表字段说明

字段名	类型	说明	安全设计意义
id	INTEGER	自增主键	内部索引
order_id	TEXT UNIQUE	UUID 订单号	随机生成，防顺序枚举
user_id	INTEGER	关联用户内部 id	外键引用 users.id，不暴露 PID
token	TEXT	订单 Token	HMAC-SM3 认证凭证，入房校验用
token_timestamp	TEXT	Token 生成时间戳	用于 Token 有效期校验
status	TEXT	订单状态	枚举值：created/taken/delivering/completed/cancelled
note	TEXT	用户备注/标签	对骑手端和管理员端隐藏，仅用户自查
created_at	TIMESTAMP	订单创建时间	用于历史查询与审计

消息表 (messages)

消息表存储通信记录，字段设计如表 7 所示。

表 7: messages 表字段说明

字段名	类型	说明	安全设计意义
id	INTEGER	自增主键	内部索引，也用于消息顺序
msg_id	TEXT UNIQUE	UUID 消息号	全局唯一，防重复提交
order_id	TEXT	关联订单号	外键引用 orders.order_id
sender_pid	TEXT	发送方 PID	不暴露真实用户名
role	TEXT	发送方角色	枚举： user/rider/admin/system
content	TEXT	消息密文	SM4-CBC 加密存储，不存明文
message_hash	TEXT	消息摘要	SM3 对明文字段联合计算，用于 Merkle Tree 叶子节点
timestamp	TEXT	消息时间戳	ISO 8601 格式，参与哈希计算
created_at	TIMESTAMP	写入时间	与 timestamp 一致性校验参考

审计日志表 (audit_logs)

审计日志表存储 Merkle Root 快照和管理员操作记录，字段设计如表 8 所示。

表 8: audit_logs 表字段说明

字段名	类型	说明	安全设计意义
id	INTEGER	自增主键	内部索引
order_id	TEXT	关联订单号	可为 NULL（登录等全局操作）
action	TEXT	操作类型	枚举： ROOT_UPDATED / VERIFY_OK / FAIL 等
detail	TEXT	操作详情（JSON）	记录消息数、不匹配哈希列表、管理员用户名等
merkle_root	TEXT	当次 Merkle Root	快照值，供后续比对使用
created_at	TIMESTAMP	操作时间	审计时间线依据

表关系与隐私设计原则

如表 5、表 6、表 7和表 8所示，系统数据库由 users 用户表、orders 订单表、messages 消息表和 audit_logs 审计日志表组成。四张表之间通过外键关联，整体以订单号为核心索引构建数据流。users 与 orders 通过 user_id 关联，orders 与 messages 通过 order_id 关联，audit_logs 通过 order_id 与订单和消息建立审计关系。

在隐私保护方面，数据库设计遵循以下原则：其一，消息明文不落库，content 字段存储 SM4-CBC 密文，明文只在服务端内存中短暂存在；其二，pid_r（随机数 r ）与 pid 均仅服务端维护，前端不可见，骑手端不可见；其三，note 备注字段在 API 响应中对骑手端和管理员端屏蔽，仅在用户本人查询订单时返回；其四，sender_pid 字段中存储的是 PID 而非用户名，管理员默认视图也不直接暴露 PID 对应的真实用户名。

3.7 用户注册与防机器注册机制

注册流程概述

FoodShield 的用户注册流程在普通用户名注册基础上增加了口令认证和数学验证码两道机制，以降低机器批量注册和匿名身份滥用风险。完整注册流程如下：

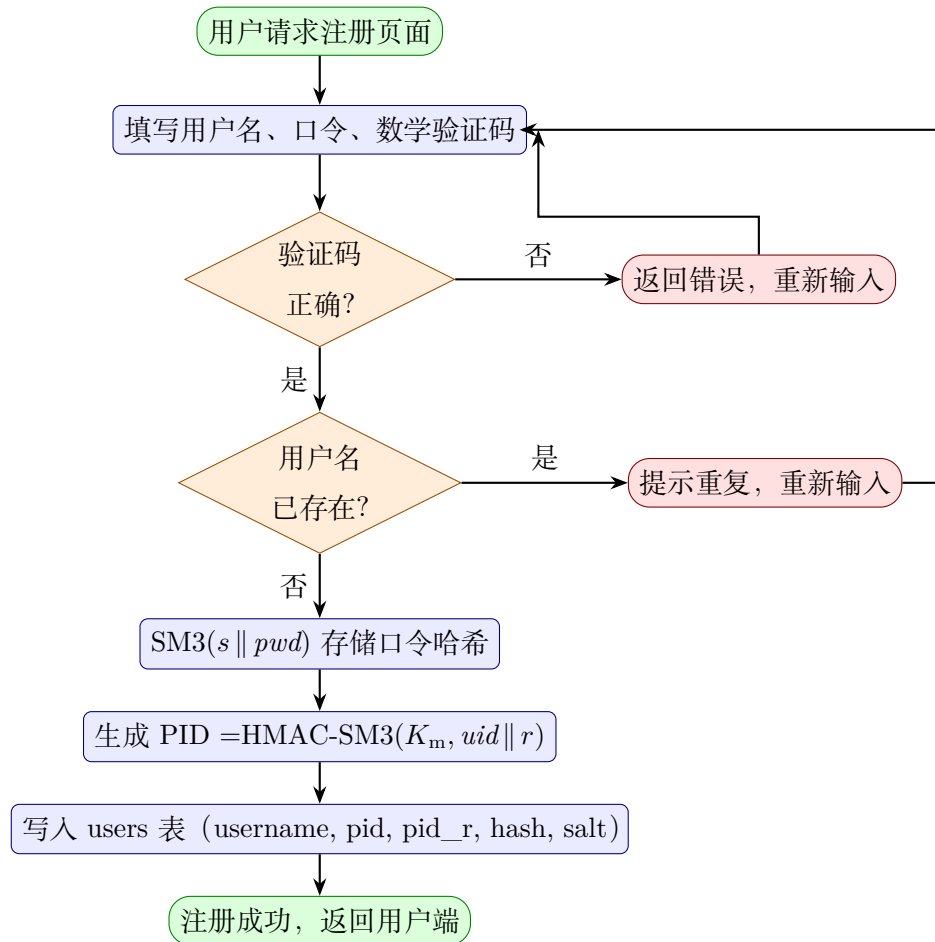


图 8: 用户注册流程图

1. 用户访问注册页面，前端向服务器请求验证码题目（GET /captcha）；
2. 服务器随机生成一道整数四则运算题，将答案存入服务端 session，向前端返回题目字符串；
3. 用户填写用户名、口令（两次确认）和验证码答案，提交注册请求（POST /register）；
4. 服务器依次执行：验证码比对、用户名唯一性校验、口令长度校验、口令一致性校验；

5. 校验通过后，生成随机盐值 s ，计算口令存储哈希：

$$h_{\text{pwd}} = \text{SM3}(s \parallel \text{password})$$

将 $\langle \text{username}, h_{\text{pwd}}, s \rangle$ 写入 `users` 表；

6. 服务器以数据库内部 `user_id` 为输入，调用 PID 生成模块，生成用户匿名身份 PID，写入 `users.pid` 和 `users.pid_r`；
7. 注册成功后自动完成登录，服务端 session 记录 `user_id` 和 `username`，前端跳转至用户主页。

口令存储机制

为保护用户口令在数据库泄露场景下的安全性，系统采用加盐哈希存储方案，而非存储明文口令。具体地，每位用户注册时，服务器使用 `secrets.token_hex(16)` 生成一个 128 位随机盐值 s ，对拼接后的字符串执行 SM3 哈希：

$$h_{\text{pwd}} = \text{SM3}(s \parallel \text{password})$$

数据库中仅保存 $\langle s, h_{\text{pwd}} \rangle$ ，原始口令在计算完成后即丢弃。登录时，服务器从数据库读取 s ，对用户提交的口令重新计算哈希，并使用恒定时间比较函数与 h_{pwd} 对比，避免时序侧信道攻击。

数学验证码机制

FoodShield 采用服务端 session 管控的数学验证码机制抵御机器批量注册攻击：

1. 每次获取验证码时，服务器从 $[1, 20]$ 范围内随机抽取两个整数，随机选择加法或减法，生成题目字符串（如 “ $13 + 7 = ?$ ”）；
2. 正确答案存入服务端 session (`session["captcha_answer"]`)，不在前端任何位置泄露；
3. 注册提交时，服务器从 session 读取答案并与用户提交值比较；比对后无论成功与否，立即从 session 中删除该答案（一次性使用），防止枚举重试；
4. 若 session 中不存在答案（如已过期或被提前消耗），注册请求直接拒绝。

该机制能够有效阻止无交互式脚本批量注册，因为攻击者需要每次实时解析题目才能通过验证，而服务端 session 的一次性特性也排除了重放答案的可能。

注册完成后的 PID 初始化

用户注册成功后，服务器以新分配的 `user_id` 为基础输入，调用 PID 生成函数，将生成的 PID 及随机数 r 分别更新至 `users.pid` 和 `users.pid_r`。从此，用户在系统中的通信身份以 PID 作为唯一标识，`username` 仅用于登录认证，不参与订单通信流程。PID 生成的详细设计见 3.8 节。

3.8 PID 匿名身份生成机制

机制概述

PID (Pseudo-Identity, 伪身份标识) 是 FoodShield 系统中用户在通信和日志中的唯一匿名代号。PID 不等同于用户名，也不等同于数据库内部 `user_id`，而是通过带密钥的哈希函数对用户标识和随机数进行混淆后得到的不可逆摘要。系统在用户注册完成后自动生成 PID，用户在通信过程中以 PID 作为发送方标识，骑手端和管理员端默认不直接展示 PID 对应的真实身份。

生成公式

PID 的生成公式为：

$$\text{PID} = \text{HMAC-SM3}(K_{\text{master}}, \text{userID} \| r)$$

其中：

- K_{master} 为系统主密钥，由服务端统一管理，不对外暴露；
- userID 为该用户在数据库中的内部整数 id（注册时自增分配）；
- r 为 128 位随机数（`secrets.token_hex(16)` 生成），每位用户独立随机，与 PID 一同保存于 `users.pid_r` 字段；
- $\|$ 表示字符串拼接；
- HMAC-SM3 为以 SM3 为底层哈希函数的消息认证码，输出 256 位（64 位十六进制字符串）。

生成时机与存储

PID 在用户首次完成注册时由服务端自动生成，写入 `users` 表。由于 PID 依赖 `user_id`，而 `user_id` 在 INSERT 操作完成后才能确定，因此注册流程采用两步写入策略：先插入临时占位 PID，获取自增 `user_id` 后再调用 PID 生成函数，将最终 PID 和随机数 r 更新至数据库。

可见性策略

PID 的对外可见性遵循“最小暴露”原则：

- **用户端**：用户仅在登录响应中短暂接收 PID（用于后续订单认证），不在任何普通业务页面长期展示。
- **骑手端**：骑手端接口响应中不包含 PID 字段，骑手无法通过正常页面获知与其通信的用户 PID。
- **管理员端**：管理员默认视图仅展示消息哈希和订单摘要，不展示 `sender_pid` 的明文值；PID 仅在条件溯源授权流程中被关联至真实用户名。
- **数据库内部**：`users.pid_r`（随机数 r ）和 `users.pid` 均保存于服务端数据库，仅服务器进程可访问，不通过 API 直接对外暴露。

抵抗跨订单攻击性

由于每位用户的 PID 由随机数 r 参与生成，不同用户之间的 PID 不存在可预测的关联关系。骑手在多次接单过程中，即使多次与同一用户通信，在缺少 K_{master} 和 r 的情况下，也无法通过 PID 判断是否为同一用户。HMAC-SM3 的伪随机性确保：在 K_{master} 保密的前提下，已知 PID 无法还原 `userID`，也无法枚举其他用户的 PID，从而实现抵抗跨订单攻击性。

3.9 订单 Token 认证机制

机制概述

订单 Token 是用户进入订单通信通道前必须提交的认证凭证。Token 由服务器在订单创建时生成，并与订单绑定保存。用户发起 WebSocket 入房请求时，服务器对 Token 的合法性、归属关系和时效性进行三重验证，缺一不可。

该机制的核心设计思路是：Token 不是一个简单的随机字符串，而是将订单号、用户 PID、时间戳和随机数绑定在一起的 HMAC-SM3 认证码。攻击者在不知道订单认证密钥 K_{order} 的情况下，无法伪造任何合法 Token，也无法从一个合法 Token 推导出其他订单的凭证。

Token 生成公式

Token 的理想生成公式为：

$$\text{Token} = \text{HMAC-SM3}(K_{\text{order}}, \text{orderID} \parallel \text{PID} \parallel \text{timestamp} \parallel \text{nonce})$$

其中：

- K_{order} 为系统订单认证密钥，由服务端统一管理；
- orderID 为随机 UUID 订单号，防止顺序枚举；
- PID 为发起订单的用户匿名身份标识；
- timestamp 为 Token 生成时的 Unix 时间戳（秒级），用于有效期校验；
- nonce 为随机数，进一步降低重放攻击风险；
- Token 生成后， $\langle \text{token}, \text{timestamp} \rangle$ 一并写入 `orders` 表供后续验证使用。

引入 nonce 的目的在于：即使攻击者能够在 Token 有效期内截获同一组（ orderID 、PID、 timestamp ）的请求，相同的时间戳配合不同的 nonce 仍会产生不同的 Token 值，从而进一步降低重放攻击成功率。

Token 验证流程

用户提交 `join_order` 请求时，服务器执行以下验证步骤：

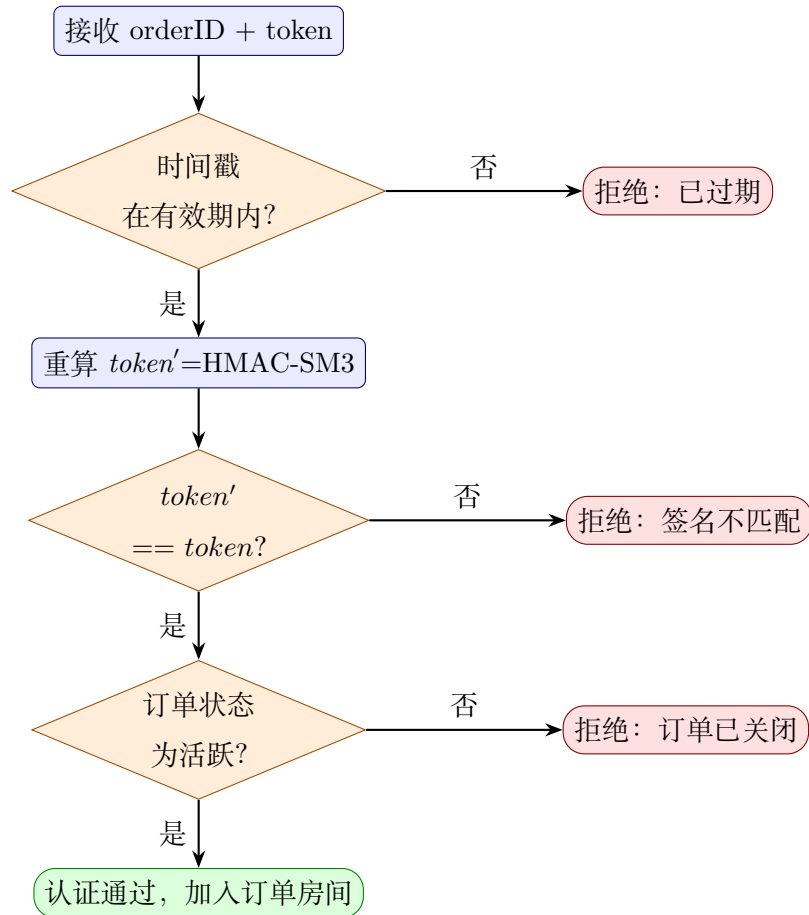


图 9: 订单 Token 三重验证流程图

1. **有效期校验**: 读取请求中的 *timestamp*, 与当前服务器时间比较, 若超出有效窗口 (默认 3600 秒), 拒绝本次请求;
2. **重新计算 Token**: 以数据库中保存的 K_{order} 、*orderID*、PID、*timestamp* (及 *nonce*) 为输入, 重新调用 HMAC-SM3 计算期望 Token;
3. **比对验证**: 将重新计算得到的期望 Token 与请求中提交的 Token 进行恒定时间比较, 一致则通过, 否则拒绝;
4. **订单状态检查**: 验证通过后进一步检查订单状态, 确保订单处于合法状态 (如未被取消), 方可允许用户加入通信房间。

安全性分析

Token 机制提供了以下安全保障:

- **抗伪造性**: Token 由 HMAC-SM3 生成, 在 K_{order} 保密的前提下, 攻击者无法在不知道密钥的情况下构造合法 Token;

- **绑定性**: Token 将 *orderId*、PID、*timestamp*、*nonce* 绑定为一个整体, 任意字段被篡改均会导致验证失败;
- **时效性**: 时间戳有效期机制限制 Token 的可用窗口, 降低截获后长期利用的风险;
- **抗重放性**: *nonce* 的引入保证即使相同的订单号和时间戳也会产生不同的 Token, 进一步降低重放成功概率。

3.10 用户端与骑手端匿名通信机制

通信通道设计

FoodShield 采用基于 Flask-SocketIO 的 WebSocket 通道实现用户端与骑手端之间的实时消息通信。系统以订单号 (*order_id*) 作为通信房间的唯一标识, 每个订单对应一个独立的 WebSocket Room, 不同订单的通信消息在房间层面实现隔离, 互不干扰。

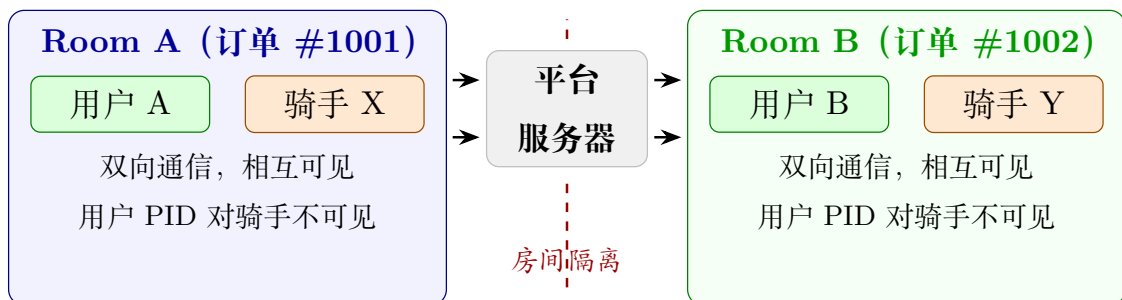


图 10: 订单房间隔离示意图 (不同订单房间成员相互不可见)

入房认证流程

用户端入房 (*join_order* 事件) 需提交以下字段:

```
{ order_id, PID, timestamp, token, role }
```

服务器按 3.9 节描述的 Token 验证流程完成三重校验 (有效期、Token 合法性、订单存在性), 验证通过后执行 *join_room(order_id)*, 将该 WebSocket 连接加入对应房间, 同时广播系统消息通知房间内成员。

骑手端入房 (join_order_as_rider 事件) 需提交:

```
{ order_id, role }
```

骑手端不需要提交 Token, 但服务器会检查订单状态, 仅当订单处于 taken、delivering 或 completed 状态时, 才允许骑手加入对应房间。这一状态约束保证只有已接单的骑手才能进入订单通信通道, 避免任意骑手随意侦听订单房间。

消息转发与字段最小化

用户端或骑手端通过 send_message 事件发送消息时, 消息负载包含:

```
{ order_id, sender_pid, role, content }
```

服务器在内存中完成以下操作后, 再将消息广播至房间:

1. 生成全局唯一 msg_id (UUID) 和 ISO 8601 格式时间戳;
2. 调用消息哈希函数计算 message_hash (详见 3.11 节);
3. 调用 SM4-CBC 对明文 content 加密, 将密文写入 messages 表 (详见 3.11 节);
4. 触发 Merkle 快照更新, 将新的 merkle_root 附加到广播消息中;
5. 通过 emit("receive_message", msg, room=order_id) 将消息广播至订单房间内所有成员。

广播给骑手端的消息中包含 sender_pid 字段, 但骑手端前端界面不展示该字段, 骑手仅能看到消息内容和发送方角色 (user 或 rider), 而无法直接接触 PID 原始值, 从而保障用户匿名性。

房间隔离与越权防护

每个订单对应一个独立房间, 服务器在处理 send_message 事件时, 还会验证订单号对应的订单是否存在。通信消息只在同一 order_id 的房间内广播, 无法跨订单传播。骑手端和用户端只能通过正常的入房鉴权流程进入自己有资格参与的订单通信房间, 无法旁听其他订单的通信内容。

3.11 消息加密存储与哈希摘要机制

消息加密存储

通信消息在写入数据库之前，经过 SM4-CBC 加密处理，数据库中不存储任何明文内容。消息处理的完整流水线如图 11 所示，加密与哈希计算并行进行，随后一并写入数据库。

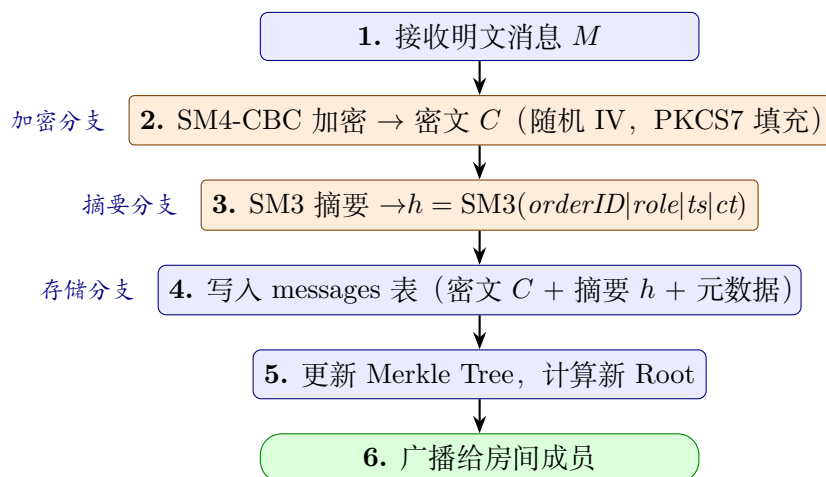


图 11: 消息处理流水线（加密存储与哈希计算顺序执行）

加密流程如下：

1. 系统从环境变量 FOODSHIELD_SM4_KEY 读取 SM4 密钥（128 位），若未配置则使用系统内置默认密钥；
2. 使用 `secrets.token_bytes(16)` 生成随机 IV（初始向量），每条消息使用独立 IV；
3. 对明文字节串执行 PKCS#7 填充，使数据长度对齐至 16 字节块边界；
4. 执行 SM4-CBC 加密，得到密文 C ；
5. 将 $IV\|C$ 拼接后以十六进制字符串形式存入 `messages.content` 字段。

存储格式可表示为：

$$\text{content} = \text{hex}(IV \parallel \text{SM4-CBC}_K(IV, M))$$

其中 IV 占前 16 字节 (32 位十六进制字符), 后续为密文。解密时, 系统从 `content` 字段读取十六进制字符串, 提取前 16 字节作为 IV , 其余部分作为密文执行 SM4-CBC 解密, 并去除 PKCS#7 填充后还原明文。

每条消息使用独立随机 IV 的设计保证了相同明文在不同时间加密后产生不同密文, 避免攻击者通过密文相等推断消息内容是否相同。

消息哈希摘要计算

每条消息在写入数据库前, 服务端在内存中对消息的明文字段进行联合哈希计算, 生成 `message_hash`, 该哈希值作为 Merkle Tree 的叶子节点参与完整性验证。

消息哈希公式为:

$$h_i = \text{SM3}(\text{orderId} \parallel \text{senderPID} \parallel \text{role} \parallel \text{content}_{\text{plain}} \parallel \text{timestamp})$$

其中各字段之间使用分隔符 `|` 拼接, 防止字段边界歧义。`contentplain` 为消息明文, 哈希计算在服务端内存中完成, 计算完成后明文即被 SM4 加密替换, 数据库中不保存明文摘要。

上述设计满足以下安全目标:

- **内容绑定:** 哈希值绑定了订单号、发送方身份、角色、明文内容和时间戳, 任意字段被篡改都会导致重新计算的哈希与存储值不一致;
- **明文隔离:** 哈希基于明文计算, 但数据库中仅存储密文, 攻击者即使获取数据库内容, 也无法从 `content` 字段直接还原哈希输入;
- **完整性可验证:** 后续验证时, 服务端先对 `content` 字段执行 SM4 解密得到明文, 再按相同公式重算哈希, 与存储的 `message_hash` 对比, 实现单条消息层面的完整性验证。

管理员访问控制

管理员查询消息记录时, 消息查询接口仅返回 `msg_id`、`order_id`、`sender_pid`、`role`、`message_hash` 和 `timestamp` 字段, 不返回 `content` 密文字段。Merkle 完整性验证只需 `message_hash` 列表, 管理员无需接触消息密文即可完成日志验证, 实现审计与明文访问的权限隔离。

3.12 Merkle 日志完整性验证机制

叶子节点构造

Merkle Tree 以订单内全部消息的哈希值列表作为叶子节点，叶子节点 L_i 即为第 i 条消息的 `message_hash`：

$$L_i = h_i = \text{SM3}(\text{orderId} \parallel \text{senderPID}_i \parallel \text{role}_i \parallel \text{content}_{i,\text{plain}} \parallel \text{timestamp}_i)$$

叶子节点按消息写入顺序排列（`timestamp ASC`，`id ASC`），顺序的固定性保证任意消息重排均会导致 Merkle Root 改变。

树的构建规则

Merkle Tree 采用迭代方式自底向上构建：

1. 以所有叶子节点哈希值的小写十六进制表示作为第 0 层；
2. 每层相邻两个节点两两拼接后计算 SM3 哈希，得到父节点：

$$\text{parent}(i) = \text{SM3}(L_{2i} \parallel L_{2i+1})$$

3. 若当前层节点数为奇数，最后一个节点与自身拼接（复制）后参与父节点计算；
4. 重复上述过程直至仅剩一个节点，该节点即为 Merkle Root R ；
5. 若订单内无消息，Root 取固定值 `SM3("EMPTY")`。

快照触发时机

Merkle Root 快照写入 `audit_logs` 表的触发时机包括以下几种：

- **实时快照**：每条消息发送成功并写入数据库后，服务器立即重新计算 Root，将 `MERKLE_ROOT_UPDATED` 记录写入 `audit_logs`；
- **管理员手动快照**：管理员可通过管理端接口手动触发某订单的 Merkle Root 快照；
- **历史补全快照**：管理员可对所有尚未生成快照的历史订单批量补全 Root 快照。

完整性验证流程

管理员执行日志完整性验证时 (POST /admin/verify/{order_id}), 系统按以下步骤完成验证:

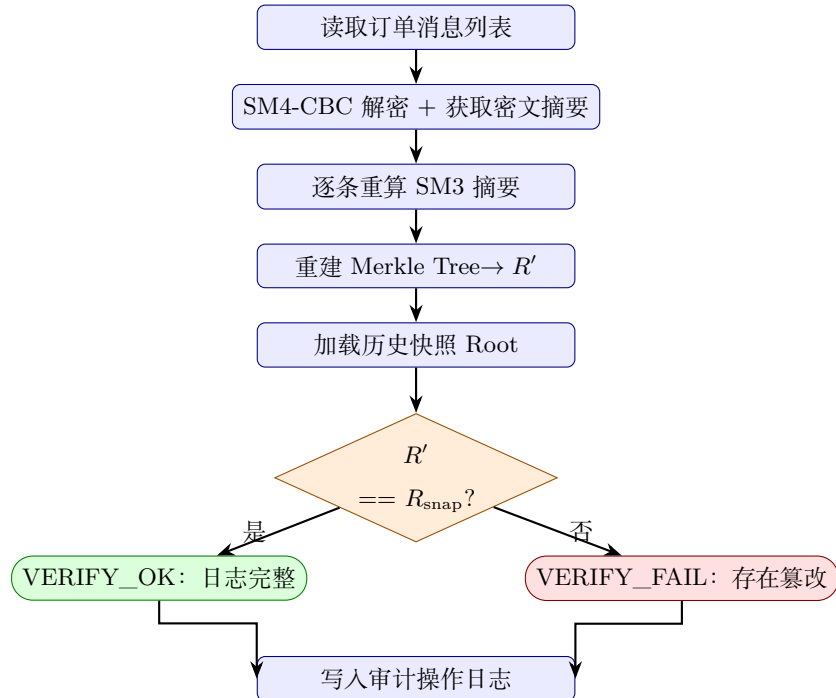


图 12: Merkle 日志完整性验证流程图

1. 从数据库按序读取订单内所有消息，对 `content` 字段执行 SM4 解密，还原明文；
2. 对每条消息按相同公式重新计算消息哈希 h'_i ，与数据库中存储的 `message_hash` 对比；若存在不匹配的条目，记录差异列表；
3. 以重新计算的哈希序列 $[h'_1, h'_2, \dots, h'_n]$ 为叶子节点重建 Merkle Tree，得到当前 Root R' ；
4. 从 `audit_logs` 读取最近一次 `MERKLE_ROOT_UPDATED` 快照，取其 `merkle_root` 字段作为历史快照值 R_{snap} ；
5. 比较 $R' \stackrel{?}{=} R_{snap}$ ：若一致且无哈希不匹配条目，则判定完整性验证通过 (VERIFY_OK)；否则判定为验证失败 (VERIFY_FAIL)；
6. 将验证结果（包括不匹配哈希列表、当前 Root、快照 Root 和消息数量）写入 `audit_logs`。

该机制能够检测以下篡改行为：

- 修改某条消息的内容或时间戳：重新计算的 h'_i 将与存储的 `message_hash` 不一致，且 $R' \neq R_{\text{snap}}$ ；
- 删除某条消息：叶子节点数量减少，树结构改变， $R' \neq R_{\text{snap}}$ ；
- 插入伪造消息：新增叶子节点， $R' \neq R_{\text{snap}}$ ；
- 重排消息顺序：叶子节点顺序改变， $R' \neq R_{\text{snap}}$ 。

3.13 条件溯源与管理员审计机制

设计原则

FoodShield 的条件溯源机制以“投诉驱动、完整性前置、关键词双重门槛”为核心设计原则。管理员不能随意对任意 PID 发起溯源，而必须在满足以下三个条件后才能完成真实身份解析：（1）提供有效订单号；（2）底层日志完整性验证通过；（3）消息内容命中指定违规关键词。

溯源申请流程

条件溯源的完整流程如图 13 所示，包含“完整性前置”与“关键词命中”两道不可绕过的门槛：

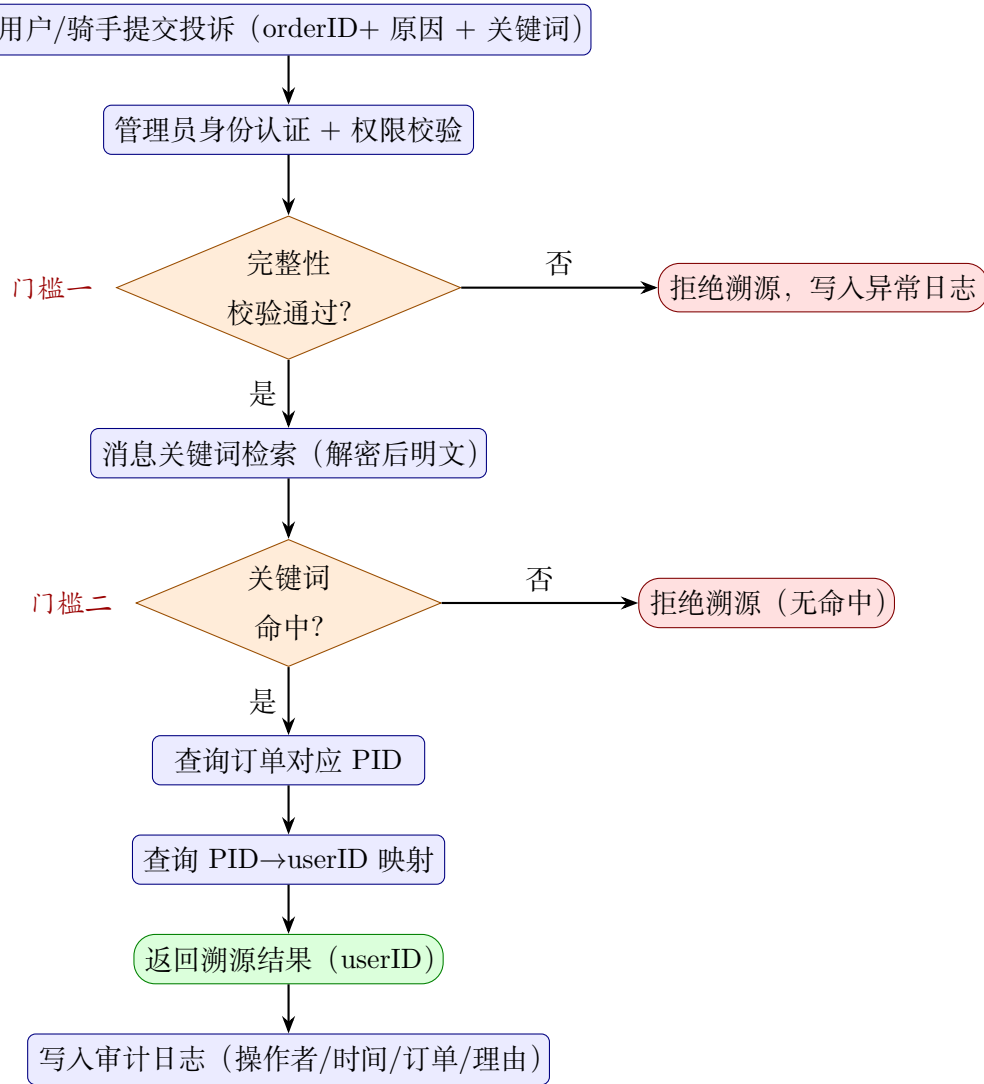


图 13: 条件溯源详细流程图 (含完整性前置校验与关键词双重门槛)

1. **申请提交**: 用户端或骑手端可在订单出现异常 (骚扰、违规发言、恶意投诉等) 情况下, 提交 $\langle order_id, keyword, reason \rangle$ 申请溯源;
2. **完整性前置校验**: 管理员端收到申请后, 服务器首先调用 `verify_order_integrity` 对目标订单执行 Merkle 完整性验证 (3.12 节)。若验证失败, 系统判定证据链已被污染, 阻断后续溯源操作并返回告警;
3. **关键词命中检测**: 完整性校验通过后, 服务器对该订单内所有消息的明文 `content` 进行关键词匹配, 收集包含指定关键词的消息发送方 PID 集合 $\mathcal{P}_{hit}$;
4. **PID 解析**: 若 $\mathcal{P}_{hit} \neq \emptyset$, 服务器在 `users` 表中查询各 PID 对应的 `username`, 得到真实用户身份列表;
5. **审计日志记录**: 无论溯源成功与否, 操作结果 (操作者、目标订单、PID、关键词、

溯源状态)均写入 `audit_logs`, 动作类型分别为 `TRACE_SUCCESS` 或 `TRACE_FAIL`;

6. **结果返回**: 管理员端展示涉嫌违规用户的 PID 和用户名, 并提示下一步处理建议。

管理员权限边界

管理员端的权限访问受以下机制约束:

- **登录认证**: 管理员需通过登录接口完成身份认证, 验证用户名和口令后才能访问管理端系列接口;
- **装饰器鉴权**: 所有管理员接口通过 `@admin_required` 装饰器拦截, 未经认证的请求将被拒绝并记录 `TRACE_DENIED` 审计日志;
- **默认隐藏明文**: 消息查询接口只返回消息哈希和元数据, 不返回消息密文, 管理员无法通过常规查询接口读取通信明文;
- **操作全程记录**: 管理员登录、登出、查询、验证、溯源成功和溯源失败等操作均写入 `audit_logs`, 使管理员行为本身也处于可追踪的审计视野中。

双重门槛的安全意义

条件溯源采用“完整性前置 + 关键词命中”双重门槛, 而不是允许管理员直接按 PID 查询真实身份, 其安全意义在于:

- 若仅依靠管理员主观判断, 溯源接口可能被滥用为隐私查询工具, 导致用户即使没有违规也可能被追踪;
- 完整性前置校验保证在证据链被污染的情况下不会基于伪造日志执行溯源, 保护被追责方的合法权益;
- 关键词命中门槛要求违规行为在消息内容中留有可验证痕迹, 避免基于推测或主观意见发起溯源。

3.14 三端摘要一致性增强机制

设计动机

在标准的 Merkle Tree 完整性验证方案中, Merkle Root 快照保存在服务端数据库 (`audit_logs` 表) 中。若攻击者同时获得数据库写权限, 理论上可以在篡改消息

内容后同步伪造 `audit_logs` 中的快照 Root，使得完整性验证仍然通过。

为提高攻击者同时伪造日志和快照的难度，FoodShield 引入三端摘要一致性增强机制：服务端在每次 Merkle Root 更新后，通过 WebSocket 消息将最新 `merkle_root` 推送给订单房间内的用户端和骑手端，前端将其存储于本地（`localStorage`）；审计时，管理员将三方 Root（服务端快照、用户端上报、骑手端上报）进行三角比对，任一方不一致则触发告警。

工作流程

1. **Root 随消息广播**：每条消息成功入库后，服务器生成最新 Root，并将其附加在消息事件响应中推送至房间：

$$\text{msg.merkle_root} \leftarrow \text{MerkleRoot}([h_1, h_2, \dots, h_n])$$

2. **客户端本地留存**：用户端和骑手端前端在收到每条消息时，将 `merkle_root` 字段保存至 `localStorage`（以 `order_id` 为键），本地仅保存最新值；
3. **主动上报**：通信结束或管理员发起审计前，用户端和骑手端可向上报接口提交本地保存的 Root：

$$\{ \text{order_id}, \text{merkle_root}, \text{role} \} \rightarrow \text{audit_logs.action} = \text{CLIENT_ROOT_REPORTED}$$

4. **三方比对**：管理员执行验证时，服务器在完成本地 Merkle 验证后，额外从审计日志中读取用户端和骑手端最近一次上报的 Root，进行三角比对：

$$R_{\text{server}} \stackrel{?}{=} R_{\text{user}} \stackrel{?}{=} R_{\text{rider}}$$

若三者全部一致，`all_match = true`；否则返回 `all_match = false` 并告警。

安全性提升

三端一致性机制将完整性验证的信任锚扩展至通信双方的本地留存，使攻击者需要同时满足以下条件才能伪造一致结果：

- 修改服务端 `messages` 表中的消息内容；
- 同步伪造 `audit_logs` 中的 Merkle Root 快照；

- 同时控制用户端和骑手端本地存储，或阻止其上报真实 Root。

三个条件需同时满足，实际攻击难度大幅提升。需要说明的是，该机制以“可选增强”而非“强制前置条件”的方式实现：若客户端未上报 Root（如通信异常中断），服务端仍可基于本地快照独立完成完整性验证；三方比对仅在客户端 Root 存在时才执行额外比对步骤。

3.15 方案安全性分析

本节对 FoodShield 的安全设计进行半形式化分析，依次针对 3.4 节所列的八项安全目标，说明系统采用的对应机制如何在威胁模型假设下提供安全保障。

目标 1：匿名性

目标：骑手端和外部攻击者在日常通信中无法从系统返回的数据中确定用户真实身份或将多次通信关联至同一用户。

分析：系统为每位用户生成 $PID = \text{HMAC-SM3}(K_{\text{master}}, userID || r)$ 。在 K_{master} 保密的前提下，HMAC-SM3 具有伪随机性，使得 PID 在计算上不可区分于随机字符串，攻击者无法从 PID 还原 $userID$ 。骑手端接口响应不暴露 PID 字段，管理员默认视图不展示 PID 与用户名的映射，普通业务流程中没有任何接口将 $(PID, username)$ 同时返回给骑手或外部调用方。

复杂度估计：在不知道 K_{master} 的情况下，穷举攻击 HMAC-SM3 需要 2^{256} 次查询，超出实际计算能力。

目标 2：认证性

目标：只有持有合法 Token 的订单参与方才能进入对应订单通信通道，攻击者无法伪造或复用 Token 冒充合法用户。

分析：Token 公式为 $\text{HMAC-SM3}(K_{\text{order}}, orderID || PID || timestamp || nonce)$ 。在 K_{order} 保密的前提下，HMAC 的不可伪造性保证攻击者无法在不知道密钥的情况下构造任何有效 Token。时间戳有效期（默认 3600 秒）限制了截获 Token 后的可利用时间窗口；nonce 的引入确保即使时间戳相同，重放攻击也无法复现合法 Token。

复杂度估计：在随机预言模型下，HMAC-SM3 满足 EUF-CMA（存在不可伪造性），伪造成功概率不超过 $q/2^{256}$ ，其中 q 为查询次数。

目标 3：机密性

目标：消息内容在数据库泄露场景下不被直接读取；管理员无需接触明文即可完成审计操作。

分析：消息明文经 SM4-CBC 加密后以密文形式存储，每条消息使用独立随机 IV，相同明文在不同时间产生不同密文，防止基于密文相等进行的内容推断。管理员消息查询接口只返回 `message_hash` 和元数据字段，不返回 `content` 字段，在接口层面实现了审计与明文访问的权限分离。

SM4-CBC 安全性：SM4 是国家密码局发布的分组密码标准，密钥长度 128 位，CBC 模式在 IV 随机且保密的条件下满足 IND-CPA 安全性，穷举密钥需要 2^{128} 次操作。

目标 4：完整性

目标：任何对历史通信记录（消息内容、顺序、数量）的修改均可被检测。

分析：每条消息的 `message_hash` 绑定了订单号、PID、角色、明文内容和时间戳五个字段，SM3 的抗碰撞性保证修改任意字段后哈希值以压倒性概率改变（碰撞概率不超过 2^{-128} ）。Merkle Tree 将所有消息哈希组织为树形结构，任意叶子节点变化均会级联改变 Root 值。三端摘要一致性机制进一步将 Root 快照分散至用户端、骑手端和服务端三方留存，攻击者需同时篡改三方数据才能伪造一致结果。

目标 5：可追责性与可审计性

目标：发生纠纷时能够通过合法流程追溯违规行为；管理员操作本身也处于可追踪的审计视野中。

分析：条件溯源的“完整性前置 + 关键词双重门槛”设计使溯源操作与违规证据直接绑定，防止无故溯源；所有管理员操作（包括溯源成功、溯源失败、完整性验证结果）均写入 `audit_logs`，形成不可篡改的操作时间线（在 Merkle 保护范围内）。

目标 6：抗滥用能力

目标：防止机器批量注册生成大量匿名身份，以及利用匿名性发起垃圾消息或骚扰。

分析：注册阶段的数学验证码（服务端 session 一次性）要求攻击者每次注册都实时解析新题目，不能通过重放答案完成批量注册；口令长度下限和用户名唯一性

约束进一步增加了批量注册的操作成本；订单号与 PID 的绑定机制使每次通信都要经过 Token 认证，无状态连接无法进入通信通道。

目标 7：隐私最小化

目标：系统各端只获取完成自身功能所必需的最小信息集合。

分析：骑手端接口不返回用户 PID 和备注字段；管理员默认接口不返回消息密文和用户真实身份；`orders.note` 备注字段在骑手端和管理员端的 API 响应中被屏蔽；`users.pid_r`（随机数）不通过任何前端接口暴露。字段最小化从数据结构和接口设计两个层面共同实现隐私最小化原则。

综合评估

综合来看，FoodShield 通过 HMAC-SM3 订单认证、SM4-CBC 加密存储、SM3 消息摘要、Merkle Tree 完整性验证、条件溯源约束和三端摘要一致性增强等机制的组合，在外卖订单通信场景中实现了匿名性、认证性、机密性、完整性、可追责性、可审计性和抗滥用能力的多目标安全保障。系统的安全设计以订单为核心索引，将业务流程与密码学机制紧密耦合，避免了安全机制游离于业务逻辑之外导致的防护盲区。

第四章 作品实现

本章围绕 FoodShield 系统的工程实现过程展开说明，重点介绍系统开发环境、后端服务、用户端、骑手端、管理员端、订单认证流程、WebSocket 通信流程、Merkle 快照验证流程以及条件溯源流程的具体实现。与第三章偏重方案设计和安全机制分析不同，本章主要说明各功能模块如何在系统原型中落地，并通过运行界面展示系统的完整业务闭环。

4.1 系统开发环境

FoodShield 系统采用轻量化 Web 原型方式实现，后端基于 Python Flask 框架构建 HTTP 接口和 WebSocket 通信服务，前端采用 HTML、CSS 和 JavaScript 实现用户端、骑手端和管理员端页面，数据库采用 SQLite 存储用户、订单、通信消息和审计日志数据。系统整体开发和测试均在本地环境完成，便于快速迭代、联调和演示。

系统主要开发环境如表 9 所示。

表 9: FoodShield 系统开发环境

项目	配置
操作系统	Windows 11
开发工具	Visual Studio Code
后端语言	Python 3.x
后端框架	Flask、Flask-SocketIO
数据库	SQLite
前端技术	HTML、CSS、JavaScript
通信机制	WebSocket
密码算法	HMAC-SM3、SM3、SM4-CBC
运行方式	<code>python -m project.server.app</code>
访问地址	<code>http://127.0.0.1:5000</code>

系统启动时，后端首先执行数据库初始化操作，检查 `users`、`orders`、`messages` 和 `audit_logs` 等核心数据表是否存在。若数据表不存在，则自动创建相关表结构；若数据表已存在，则直接加载现有数据并启动服务。启动完成后，Flask 服务监听本

地 5000 端口，用户可通过浏览器访问用户端、骑手端和管理员端页面。

```

O (venv) PS C:\Users\ASUS\Desktop\FoodShield> python -m project.server.app
Database initialized successfully.
[FoodShield config warning] default demo secrets are in use: FOODSHIELD_SECRET_KEY, FOODSHIELD_ADMIN_USERNAME, FOODSHIELD_ADMIN_PASSWORD, FOODSHIELD_ORDER_KEY, FOODSHIELD_MASTER_KEY, FOODSHIELD_ORDER_ALIAS_SECRET, FOODSHIELD_SM4_KEY
Set these environment variables before production or public-network demos.
* Serving Flask app 'app'
* Debug mode: on
WARNING: This is a development server. Do not use it in a production deployment. Use a production WSGI server instead.
* Running on http://127.0.0.1:5000
Press CTRL+C to quit
* Restarting with stat
Database initialized successfully.
[FoodShield config warning] default demo secrets are in use: FOODSHIELD_SECRET_KEY, FOODSHIELD_ADMIN_USERNAME, FOODSHIELD_ADMIN_PASSWORD, FOODSHIELD_ORDER_KEY, FOODSHIELD_MASTER_KEY, FOODSHIELD_ORDER_ALIAS_SECRET, FOODSHIELD_SM4_KEY
Set these environment variables before production or public-network demos.
* Debugger is active!
* Debugger PIN: 336 338 770

```

图 14: 系统后端服务启动界面

4.2 后端服务实现

后端服务是 FoodShield 系统的核心控制单元，主要负责用户注册登录、PID 生成、订单创建、Token 验证、消息转发、消息加密存储、Merkle Root 快照生成、完整性验证和条件溯源等功能。后端服务整体上可以划分为业务接口层、安全机制层、数据访问层和审计控制层。

业务接口层主要面向用户端、骑手端和管理员端提供 HTTP API。例如，用户端通过注册接口完成账号创建，通过登录接口完成身份认证，通过创建订单接口生成订单号和订单 Token；骑手端通过订单查询和接单接口获取可参与的订单；管理员端通过登录、查看日志、生成快照、完整性验证和条件溯源接口完成审计操作。

安全机制层负责实现系统的核心密码学功能，包括 PID 匿名身份生成、HMAC-SM3 订单 Token 生成与验证、SM4-CBC 消息加密存储、SM3 消息摘要计算、Merkle Tree 构建和 Root 快照比对。后端在处理业务请求时，会根据不同业务场景调用相应的安全模块。例如，用户创建订单时调用 Token 生成模块，用户加入订单房间时调用 Token 验证模块，用户或骑手发送消息时调用加密和哈希模块，管理员验证日志时调用 Merkle Tree 验证模块。

数据访问层封装了对 SQLite 数据库的读写操作。系统主要通过 `users` 表保存用户登录信息和 PID 映射关系，通过 `orders` 表保存订单号、订单状态、用户关联关系和 Token，通过 `messages` 表保存订单通信消息密文和消息摘要，通过 `audit_logs` 表保存 Merkle Root 快照、完整性验证结果和管理员操作记录。

审计控制层主要服务于管理员端。管理员执行登录、查看消息摘要、生成快照、

验证日志、处理溯源申请等高权限操作时，系统会将操作类型、目标订单、操作结果和时间写入 `audit_logs` 表。该设计使管理员行为本身也处于审计范围内，避免管理员权限成为新的隐私泄露风险。

系统主要后端接口和 WebSocket 事件如表 10 所示。

表 10: 后端主要接口与事件

接口或事件	功能说明	主要安全机制
<code>/register</code>	用户注册	口令哈希、验证码、PID 生成
<code>/login</code>	用户登录	口令校验、Session
<code>/create_order</code>	创建订单	绑定 PID、生成订单 Token
<code>/orders</code>	查询订单历史	用户身份校验
<code>join_order</code>	用户加入订单房间	<code>order_id</code> 与 Token 校验
<code>join_order_as_rider</code>	骑手加入订单房间	订单状态校验
<code>send_message</code>	发送订单消息	SM4 加密、SM3 摘要、Merkle 更新
<code>/admin/login</code>	管理员登录	管理员身份认证
<code>/admin/messages/{order_id}</code>	查看消息摘要	默认不返回明文
<code>/admin/snapshot/{order_id}</code>	生成 Merkle 快照	Root 写入审计日志
<code>/admin/verify/{order_id}</code>	验证日志完整性	Merkle Root 比对
<code>/admin/trace</code>	执行条件溯源	权限校验、完整性前置、审计记录

通过上述接口和事件设计，FoodShield 将普通业务流程与安全机制进行紧密绑定。用户和骑手只能围绕订单号进行通信，管理员只能在认证后访问审计功能，所有敏感操作均由后端统一控制。

4.3 用户端功能实现

用户端主要面向外卖平台中的普通消费者，提供注册登录、创建订单、订单历史查询、订单搜索、订单备注、订单通信和投诉申请等功能。用户端的设计目标是在保证用户能够顺利完成下单和通信的基础上，隐藏用户真实身份，避免将真实用户

名、联系方式或 PID 直接暴露给骑手端。

用户首次使用系统时，需要在注册页面输入用户名、口令和验证码。服务器完成验证码校验、用户名唯一性检查和口令哈希存储后，为该用户生成 PID 匿名身份标识。PID 作为系统内部身份标识参与后续订单绑定和消息记录，但普通页面不长期展示 PID，骑手端也无法通过正常业务接口获取用户 PID。

The image displays a user registration interface. At the top, a green header contains the text '用户端 · 匿名订单认证' and '匿名下单与可信 Token 验证'. Below this, a paragraph explains the process: '从注册登录、创建订单到进入匿名聊天房间，用户端始终不展示 PID；身份绑定、订单认证与审计 Root 均由后端协同完成。'. There are four green boxes with white text: 'PID 后端托管' (User interface does not expose real identity mapping), '订单 Token' (Verify order attribution and篡改 risk), '私有订单历史' (Remarks and tags are only visible to the user interface), and 'Root 上报' (Supports three-party consistency verification for management staff). Below the header is a white registration form titled '1. 登录 / 注册'. It prompts the user to '请先登录或注册。' and contains three input fields: a username field with 'aaa', a password field with '.....', and a verification code field with '3|'. A '获取验证码' button is next to the verification code field. Below the inputs is a math problem '7 - 4 = ?'. At the bottom of the form is a green '注册' button and a link '已有账号? 返回登录'.

图 15: 用户注册界面

用户登录成功后进入用户主页。用户主页提供创建订单、查看历史订单、搜索订单和进入通信页面等功能。用户创建订单时，服务器随机生成订单号，并基于订单号、用户 PID、时间戳和随机数生成订单 Token。订单创建成功后，用户端获得订单号和 Token，并可通过该凭证进入订单通信通道。

2. 创建订单

必须登录后创建订单。前端不再提交 PID，后端从 session 绑定用户。

订单备注（用户可见，管理员默认不展示）

订单标签（用户端私有检索字段）

配送说明（骑手可见，不等同于用户标签）

创建订单

订单号

时间戳

Token

订单状态

created

图 16: 用户端订单创建界面

用户进入订单通信页面时,前端携带 `order_id` 和 `token` 向服务器发起 `join_order` 事件。服务器验证 `Token` 后,将该连接加入对应订单房间。通信过程中,用户输入的消息会先发送到后端,由后端完成消息编号生成、时间戳生成、SM4 加密、SM3 哈希和 Merkle Root 更新,再广播给同一订单房间中的骑手端。



图 17: 用户端订单通信界面

在异常场景下，用户端还可以基于订单号提交溯源申请。申请内容包括订单号、投诉原因和关键词。系统不允许用户直接按 PID 进行溯源申请，而是要求用户以订单号为入口提交投诉，保证溯源流程贴合外卖订单纠纷的真实业务逻辑。

6. 提交溯源申请

用户可按订单号提交投诉/异常申请，等待管理员审批后条件溯源；不需要也不能提交 PID。

3f87f293-5e70-4750-8b0a-0066fd902bc2

未按时送达

提交溯源申请

已提交，申请编号：2，等待管理员审批。

图 18: 用户端溯源申请界面

4.4 骑手端功能实现

骑手端主要面向配送人员，提供订单查看、订单搜索、进入订单通信和异常申请等功能。骑手端的实现重点是满足配送沟通需求，同时严格限制敏感信息展示范围。与传统外卖系统中骑手可能看到用户真实姓名、联系方式或详细备注不同，Food-Shield 骑手端只展示完成配送所需的订单信息和通信入口，不展示用户真实身份，不展示用户 PID，也不展示用户备注和标签原文。

骑手进入系统后，可以查看当前可处理订单，也可以根据订单号搜索目标订单。骑手端在进入通信页面前，需要通过订单状态校验。服务器只有在订单处于可通信状态时，才允许骑手加入对应订单房间。该机制可以防止任意骑手随意进入其他订单的通信通道。



图 19: 骑手端订单管理界面

骑手加入订单通信房间后，可以与用户进行实时通信。前端页面仅展示发送方角色、消息内容和时间等必要信息，不展示用户 PID 或真实用户名。即使后端消息记录中包含发送方匿名标识，骑手端页面也不会将该字段渲染出来，从而降低骑手通过固定标识进行跨订单关联的风险。



图 20: 骑手端订单通信界面

当骑手认为订单通信中存在恶意投诉、辱骂威胁或其他异常行为时，也可以基于订单号提交溯源申请。该设计保证了溯源申请不是单向的，用户端和骑手端均可提出，管理员端根据订单号统一处理。这一点增强了系统在真实平台管理场景中的公平性和完整性。



图 21: 骑手端异常申请界面

4.5 管理员端功能实现

管理员端用于平台侧异常处理和可信审计，是 FoodShield 系统区别于普通聊天系统的重要组成部分。管理员端不是普通的聊天明文查看后台，而是以订单摘要、消息哈希、Merkle Root、完整性验证结果和审计日志为核心的可信审计后台。

管理员访问后台前需要先完成登录认证。系统会校验管理员用户名和口令，认证通过后在服务端 Session 中记录管理员状态。所有管理员接口均受权限校验保护，未登录请求无法访问管理员功能。管理员登录、登出、查看订单、生成快照、执行验证和条件溯源等操作均会写入审计日志。

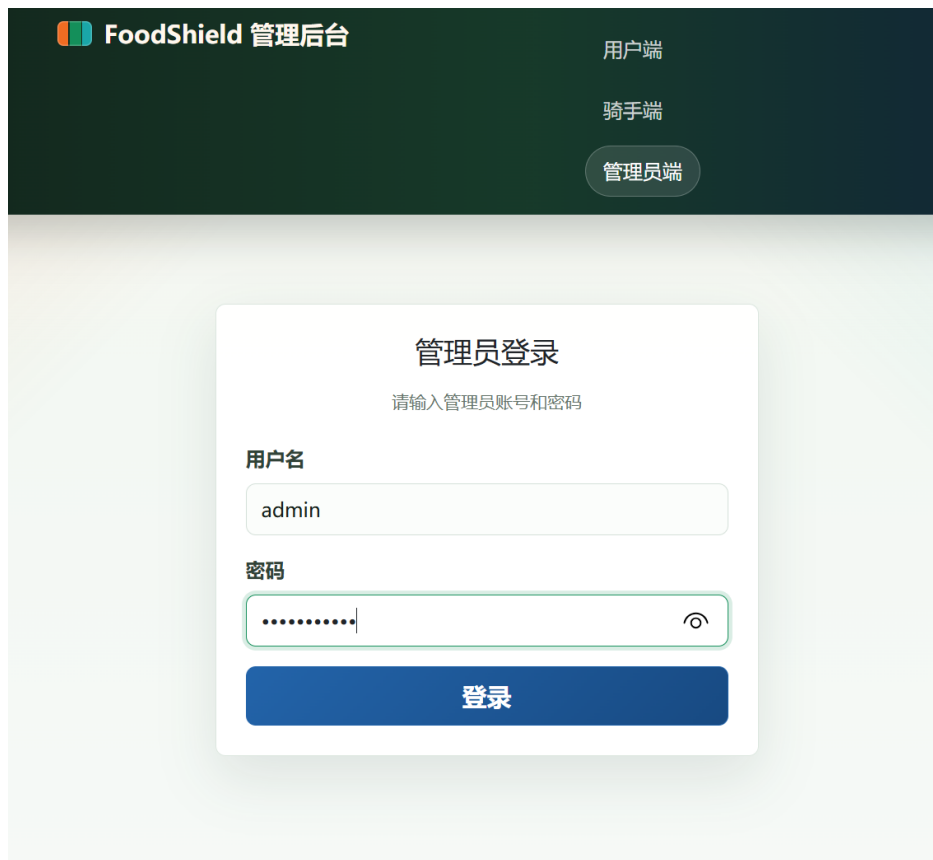


图 22: 管理员登录界面

登录后，管理员可以根据订单号检索订单摘要和通信日志。管理员默认只能查看消息编号、订单号、发送方角色、时间戳、消息哈希和 Merkle Root 等审计信息，不能通过普通查询接口直接查看通信明文。该设计将审计能力和明文查看能力分离，降低管理员权限带来的二次隐私泄露风险。



时间戳	角色	内容状态	消息 Hash
2026/7/3 00:46:20	user	Hash-only 不展示明文 / PID	90d2b9f388...c6c96a
2026/7/3 00:49:40	rider	Hash-only 不展示明文 / PID	f368e1c6d8...6e3e9f

图 23: 管理员消息摘要查看界面

管理员可以手动为某一订单生成 Merkle Root 快照。系统会读取该订单当前所有消息哈希，按照 Merkle Tree 构建规则计算 Root，并将 Root、订单号、消息数量和快照时间写入 `audit_logs` 表。后续完整性验证时，系统会重新计算当前 Root，并与历史快照 Root 进行比对。

 审计搜索

订单号

3f87f293-5e70-4750-8b0a-0066fd902bc2

消息 Hash

f368e1c6d8

审计动作

全部动作

按订单号加载 Hash 记录

搜索审计订单

管理员端仅按订单号、消息 Hash、Merkle Root 和审计动作检索。
不展示订单备注、标签、PID、Token 或通信明文。

 安全审计 (Merkle Tree)

1. 生成快照 (Snapshot)

2. 验证完整性

3. 三端 Root 一致性验证

☒ 快照生成成功

message_count: 2

Merkle Root:

5a433d74f4329e5de9f17d32f094fde1496c94bbd454727ba5c05ec67639869c

图 24: Merkle Root 快照生成界面

管理员还可以处理用户端或骑手端提交的溯源申请。管理员执行条件溯源前，系统会先对目标订单执行完整性验证，若验证失败则阻断溯源；若验证通过，再根据投诉关键词检索消息内容，并在关键词命中后完成从订单号到 PID、再从 PID 到真实用户身份的受控映射查询。溯源成功或失败均写入审计日志。



图 25: 管理员条件溯源界面

4.6 注册、下单与订单认证流程实现

FoodShield 的注册、下单与订单认证流程构成系统业务闭环的前半部分, 其主要目标是完成用户身份初始化、订单身份绑定和通信资格认证。

用户注册阶段, 前端向服务器提交用户名、口令和验证码。服务器首先验证验证码是否正确, 然后检查用户名是否重复, 并对用户口令进行加盐哈希存储。用户记录创建完成后, 服务器根据用户内部 id 和随机数生成 PID, 并将 PID 与用户记录绑定保存。该过程保证用户在系统内部拥有可控匿名身份, 而不是直接以真实用户名参与通信。

订单创建阶段, 用户登录后向服务器发起创建订单请求。服务器生成随机 UUID 形式的订单号, 将订单与当前用户的内部 id 和 PID 进行绑定, 同时生成订单 Token。Token 不是简单随机字符串, 而是基于订单号、PID、时间戳和随机数计算得到的 HMAC-SM3 认证值。生成后的 Token 保存于 `orders` 表, 并返回给用户端用于后续通信认证。

用户进入订单通信页面时, 前端需要提交 `order_id` 和 `token`。服务器根据数据库中保存的订单信息重新计算期望 Token, 并与用户提交的 Token 进行恒定时间比

较。若 Token 不匹配、订单不存在或订单状态异常，则拒绝加入订单通信房间；只有验证通过后，服务器才允许用户加入对应 WebSocket Room。

该流程可以抽象为以下流程图：

注册、下单与订单认证流程图



图 26：注册、下单与订单认证流程

图 26：注册、下单与订单认证流程

该实现使订单号不再只是普通业务编号，而成为认证、通信、日志和溯源的统一索引。攻击者即使获得某个订单号，也无法在不知道合法 Token 的情况下进入对应通信通道，从而降低订单隐私泄露风险。

4.7 WebSocket 通信流程实现

FoodShield 使用 Flask-SocketIO 实现用户端与骑手端之间的订单级实时通信。系统以订单号作为通信房间标识，每个订单对应一个独立的 WebSocket Room。用户和骑手只有通过各自的入房认证后，才能加入对应订单房间并发送或接收消息。

用户端入房时触发 `join_order` 事件，并提交 `order_id`、`token` 和角色信息。服务器校验 Token 成功后执行 `join_room(order_id)`，将当前连接加入订单房间。骑手端入房时触发 `join_order_as_rider` 事件，服务器检查订单是否存在以及订单状态是否允许通信，校验通过后将骑手连接加入同一订单房间。

消息发送时，前端触发 `send_message` 事件，将订单号、发送方角色和消息内容发送到服务器。服务器接收到消息后，依次完成以下操作：

1. 检查订单号是否有效；
2. 生成全局唯一消息编号 `msg_id`；
3. 生成 ISO 8601 格式时间戳；
4. 对消息明文执行 SM4-CBC 加密；
5. 对订单号、发送方标识、角色、明文内容和时间戳计算 SM3 消息摘要；

6. 将密文、消息摘要和元数据写入 `messages` 表；
7. 更新该订单对应的 Merkle Root 快照；
8. 将消息广播给同一订单房间内的用户端和骑手端。

通信流程可以抽象为以下流程图：

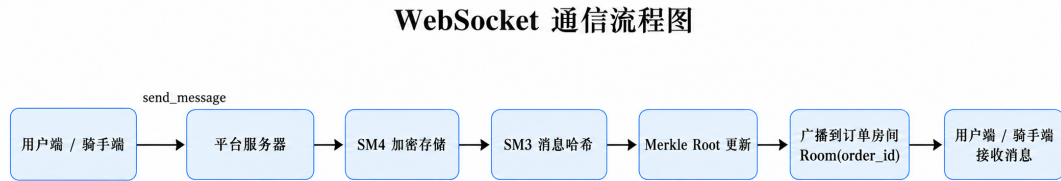


图 27: 订单级 WebSocket 通信流程

通过订单房间隔离机制，不同订单之间的消息不会相互传播。通过后端统一转发机制，系统能够在消息进入数据库前同步完成加密、哈希和审计操作，使普通聊天流程和安全机制形成联动，而不是在聊天结束后再额外补做安全处理。

4.8 Merkle 快照与完整性验证实现

Merkle 快照与完整性验证是 FoodShield 可信审计能力的核心。系统在消息写入数据库后，会根据当前订单内所有消息哈希构建 Merkle Tree，并生成 Merkle Root 快照。该 Root 代表某一时刻该订单通信记录的完整性状态。

在快照生成过程中，系统首先按照消息写入顺序读取目标订单的 `message_hash` 列表，将每条消息哈希作为 Merkle Tree 的叶子节点。然后，系统对相邻叶子节点进行拼接并计算 SM3 哈希，逐层向上构建父节点，最终得到唯一的 Merkle Root。若某一层节点数为奇数，则复制最后一个节点参与下一层计算。计算完成后，系统将订单号、消息数量、Root 值、快照时间和操作类型写入 `audit_logs` 表。



图 28: Merkle Root 快照生成结果

完整性验证时，管理员端提交目标订单号，服务器重新读取该订单当前消息记录，并重新计算每条消息的哈希值。随后，系统以重新计算得到的哈希序列构建新的 Merkle Root，并与 `audit_logs` 表中保存的历史快照 Root 进行比对。若两者一致，说明当前通信记录与快照生成时保持一致；若两者不一致，则说明消息内容、消息数量或消息顺序可能被修改。

系统能够检测以下几类篡改行为：

1. 修改消息内容：重新计算的消息哈希发生变化，导致 Root 不一致；
2. 删除历史消息：叶子节点数量变化，导致 Root 不一致；
3. 插入伪造消息：新增叶子节点，导致 Root 不一致；
4. 重排消息顺序：叶子节点顺序变化，导致 Root 不一致；
5. 修改消息元数据：时间戳、角色或订单号变化，导致消息哈希变化。

完整性验证流程可以抽象为：

完整性验证流程图



图 29: 完整性验证流程图



图 30: 日志完整性验证通过结果

此外，系统实现了三端摘要一致性增强机制。服务器在消息更新后将最新 Merkle Root 推送给用户端和骑手端，客户端可将 Root 保存在本地，并在审计时主动上报。管理员验证时可以将服务端快照 Root、用户上传 Root 和骑手端上报 Root 进行三方比对。若三者一致，则说明三端对订单通信摘要状态达成一致；若不一致，则提示可能存在日志篡改、客户端异常或上报数据不一致。



图 31: 三端摘要一致性验证结果

4.9 条件溯源流程实现

条件溯源模块用于在投诉、恶意骚扰、虚假争议或平台审计场景下恢复必要身份信息。FoodShield 的条件溯源流程以订单号为入口，而不是以 PID 为入口。用户端和骑手端均可基于同一订单号提交溯源申请，管理员端统一处理申请并执行受控溯源。

溯源申请通常包含三个字段：order_id、reason 和 keyword。其中，order_id 用于确定目标订单，reason 用于说明溯源原因，keyword 用于指示需要检索的异常内容。系统要求溯源申请围绕订单发起，而不是允许管理员或普通用户任意输入 PID 进行身份查询。该设计更符合外卖平台实际纠纷处理流程，也降低了 PID 被滥用的

风险。

管理员处理溯源申请时，系统首先执行完整性前置校验。服务器调用 Merkle 完整性验证模块，对目标订单的通信日志进行验证。若当前 Root 与历史快照 Root 不一致，系统认为该订单通信证据链可能已经被污染，拒绝继续执行溯源，并将失败原因写入审计日志。

若完整性验证通过，系统进一步根据申请中的关键词检索订单消息内容。由于数据库中保存的是 SM4-CBC 密文，服务器需要在内存中解密消息内容并执行关键词匹配。若未命中关键词，系统拒绝溯源并记录 TRACE_FAIL；若命中关键词，系统根据命中消息的发送方 PID 查询 users 表，恢复必要的用户身份信息，并将结果返回给管理员端。

条件溯源流程可以抽象为：

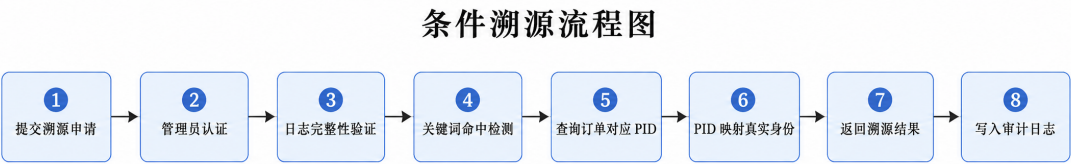


图 32: 条件溯源流程图

6. 提交溯源申请

用户可按订单号提交投诉/异常申请，等待管理员审批后条件溯源；不需要也不能提交 PID。

3f87f293-5e70-4750-8b0a-0066fd902bc2

未按时送达

提交溯源申请

已提交，申请编号：4，等待管理员审批。

图 33: 订单溯源申请提交界面



图 34: 条件溯源成功结果

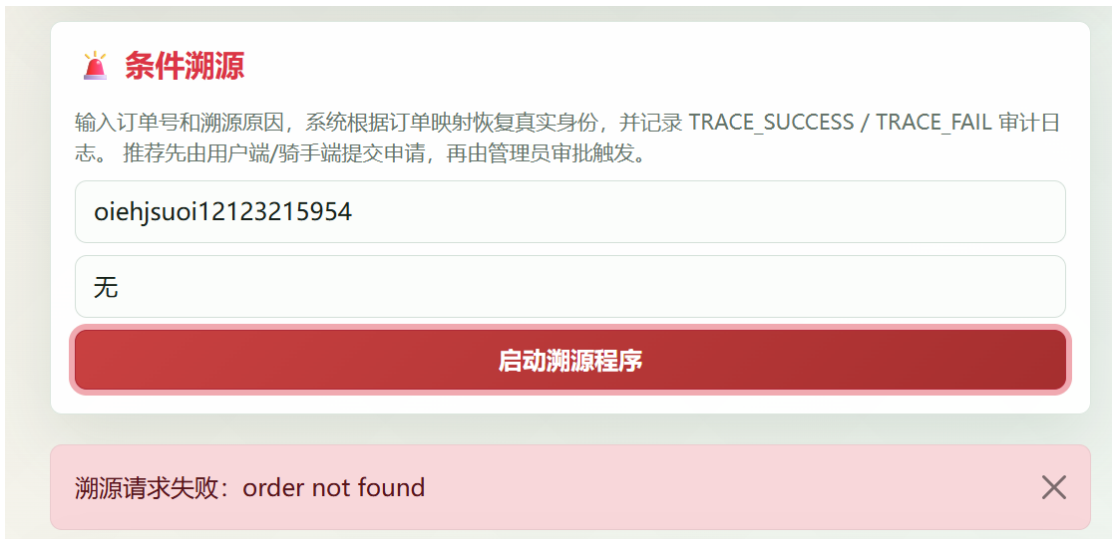


图 35: 条件溯源失败结果

该流程体现了 FoodShield 的可控匿名思想: 正常通信中不暴露真实身份, 异常场景下也不能随意溯源, 必须经过订单号入口、管理员认证、完整性验证和关键词命中等约束。这样既保留了平台管理能力, 又降低了审计权限被滥用的风险。

4.10 系统运行界面

为了验证 FoodShield 系统的完整性和可展示性，本节从用户端、骑手端和管理员端三个角度展示系统运行界面。系统运行界面覆盖注册、登录、创建订单、订单通信、管理员登录、日志查看、Merkle 快照生成、完整性验证、条件溯源和三端摘要一致性等关键功能。

首先，用户端完成注册和登录后，可以创建订单并进入订单通信页面。订单创建成功后，系统生成随机订单号和订单 Token，用户端凭借订单号和 Token 加入对应通信房间。用户端页面展示订单状态、通信窗口和消息发送区域。



图 36: 用户端功能运行界面

其次，骑手端可以查看订单并进入订单通信页面，与用户进行实时沟通。骑手端页面不展示用户真实身份和 PID，仅展示订单编号、订单状态和通信内容。该界面体现了系统的字段最小化展示原则。

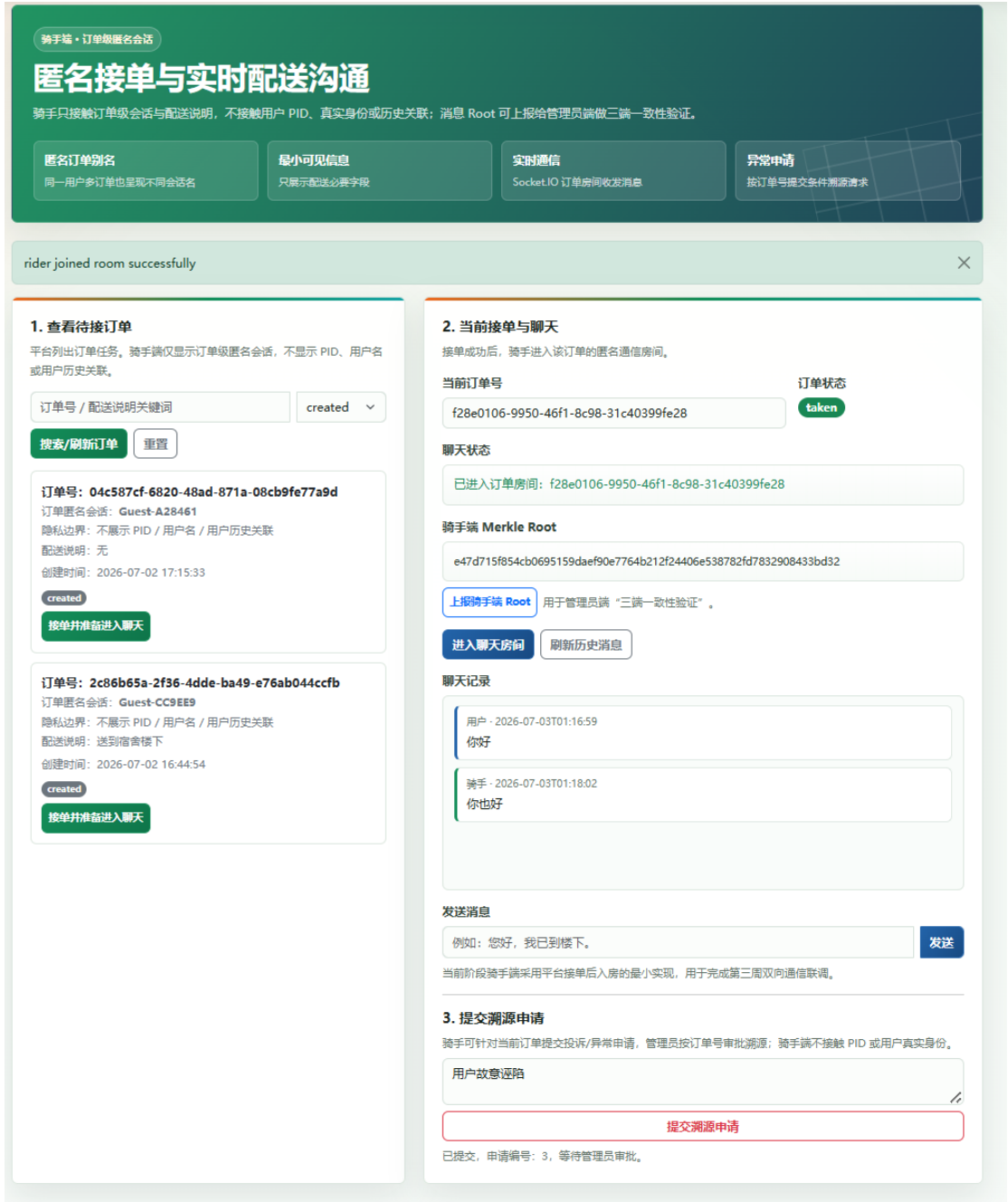


图 37: 骑手端功能运行界面

再次，管理员端完成登录后，可以基于订单号查看订单审计摘要、消息哈希和 Merkle Root。管理员可以手动生成 Root 快照，也可以执行完整性验证。若日志未被篡改，系统返回验证通过；若消息记录被修改、删除、插入或重排，系统返回验证失败并记录审计日志。



图 38: 管理员端日志审计界面

最后，在异常场景下，用户端和骑手端均可提交溯源申请，管理员端根据订单

号处理申请。系统先执行 Merkle 完整性验证，再进行关键词命中检测，最后在满足条件时返回溯源结果。该界面展示了 FoodShield 从匿名通信到可信审计再到条件溯源的完整闭环。



图 39: 条件溯源功能运行界面

通过上述运行界面可以看出，FoodShield 已经实现了用户端、骑手端和管理员端的完整业务流程。系统不仅支持普通外卖订单通信，还在通信过程中集成了匿名身份、订单认证、消息加密、日志哈希、Merkle Root 快照、完整性验证和条件溯源等安全机制。相比普通聊天系统，FoodShield 的实现更强调隐私保护、权限边界和可信审计，能够较好地支撑外卖订单通信场景下的安全需求。

第五章 作品测试与分析

本章对 FoodShield 系统进行全面测试与分析，验证系统功能完整性、安全机制有效性、日志完整性检测能力、访问控制边界以及关键路径性能。测试内容与第三章威胁模型中定义的各类攻击场景严格对应，旨在证明系统设计中的安全目标在实际实现中得到有效落地。

5.1 测试环境

FoodShield 系统的测试环境配置如表 11所示。所有测试均基于 Flask 内置测试客户端完成，使用独立的临时数据库，不修改演示数据库，确保测试结果的可复现性和数据隔离性。

表 11: 测试环境配置

项目	配置
操作系统	Arch Linux (WSL2, Kernel 6.18.33.1)
CPU	Intel i5-13420H (WSL2 分配 2 核 4 线程, 2.611 GHz)
内存	7.8 GiB
Python 版本	3.13
国密算法库	gmssl
数据库	SQLite 3
测试框架	Flask test client (隔离数据库)

测试方法分三类：功能测试（正向调用各业务接口验证流程正确性）、安全测试（攻击演示脚本 `attack_demo.py` 模拟各类威胁场景）、性能测试（基准脚本在不同数据规模下收集关键路径耗时）。

5.2 功能测试

本节对 FoodShield 系统的核心功能进行正向测试，覆盖用户注册、PID 生成、口令认证、订单创建与 Token 生成、Token 验证、骑手接单、双向实时通信、消息加密存储、Merkle 快照生成、条件溯源和三端一致性验证等关键功能。测试结果如表 12所示，14 项测试全部通过。系统各端运行界面截图详见第四章 4.10 节。

表 12: 功能测试用例与结果

编号	测试项	输入描述	预期结果	实际	结论
F01	用户注册	用户名 + 密码 + 验证码	生成 PID	通过	✓
F02	PID 生成	注册后自动	前端不展示	通过	✓
F03	用户登录	用户名 + 密码	SM3 验证通过	通过	✓
F04	创建订单	登录用户	生成 orderID+Token	通过	✓
F05	Token 验证	合法 Token	valid=true	通过	✓
F06	Token 过期	超时时间戳	valid=false	通过	✓
F07	骑注册	骑手信息	生成 rider_pid	通过	✓
F08	骑手接单列表	骑手登录	不展示 PID	通过	✓
F09	骑手接单	骑手 +orderID	状态变更	通过	✓
F10	实时通信	同订单房间	消息收发	通过	✓
F11	加密存储	发送消息	content 为密文	通过	✓
F12	Merkle 快照	管理员触发	Root 入审计日志	通过	✓
F13	条件溯源	用户申请 + 审批	返回用户信息	通过	✓
F14	三端一致性	管理员验证	三方 Root 一致	通过	✓

其中, Token 生成采用 $\text{HMAC-SM3}(K_{\text{order}}, \text{orderID} \parallel \text{PID} \parallel \text{timestamp} \parallel \text{nonce})$, 验证时后端从 session 获取 PID, 不信任前端提交值。消息写入前经 SM4-CBC 加密 (独立 IV), 数据库存储 IV + ciphertext。三端一致性验证比对管理员、用户和骑手各自上报的 Merkle Root, 三者完全一致方判定通过。

5.3 安全性测试

本节使用攻击演示脚本 (attack_demo.py) 针对第三章定义的各类威胁场景进行安全测试。脚本包含 11 个场景 (含 2 个正常操作基线), 测试结果如表 13 所示, 完整运行输出如图 40 所示。

表 13: 安全性测试用例与结果

编号	攻击类型	威胁	攻击手段	预期防御	实际	结论
S01	未登录创建订单	T1 伪造	无登录态调用创建订单	401 拒绝	401	✓
S02	伪造 Token	T2 伪造	修改 Token 末位字符	valid=false	valid=false	✓
S03	跨用户越权	T1 越权	mallory 验证 alice 订单	403 拒绝	403	✓
S04	未登录接单	T1 伪造	无登录态调用接单	401 拒绝	401	✓
S05	正常接单（基线）	—	骑手首次接单	200 成功	200	✓
S06	重复接单	T6 滥用	同一骑手再次接单	400 拒绝	400	✓
S07	管理员越权	T6 越权	无登录态访问管理员接口	403+ 审计日志	403	✓
S08	管理员登录（基线）	—	demo 账号登录	200 成功	200	✓
S09	hash-only 审计	T7 隐私	管理员查看消息接口	无 content 字段	无 content	✓
S10	篡改检测	T4 完整性	修改 DB 消息密文	VERIFY_FAILmismatch=1		✓
S11	溯源滥用阻断	T5 追责	调用明文关键词溯源	403 阻断	403	✓

```

[m. / FoodShield]$ uv run python -m project.tools.attack_demo
Database initialized successfully.
FoodShield attack demo
database: /tmp/foodshield_attack_c9f8251cd7ee46cc8b7618526dbd41ff.db

[unauthenticated_create_order]
expected: 401 because order creation requires a logged-in user session
actual: status=401, success=False, message=请先登录后再操作

[forged_token]
expected: valid=false because HMAC-SM3 token was modified
actual: status=200, success=True, message=order verification completed
detail: {'data': {'valid': False}}

[cross_user_verify]
expected: 403 because another logged-in user cannot verify Alice's order
actual: status=403, success=False, message=只能验证当前登录用户自己的订单

[unauthenticated_rider_take_order]
expected: 401 because rider must register/login before taking orders
actual: status=401, success=False, message=请先登录骑手账号后再操作

[first_take_order]
expected: 200 because the logged-in rider takes an unassigned created order
actual: status=200, success=True, message=order taken successfully
detail: {'data': {'order_id': 'b76e4076-32de-4347-9c50-d59f5e7b094e', 'status': 'taken'}}

[duplicate_take_order]
expected: 400 because the order has already been taken
actual: status=400, success=False, message=order cannot be taken, current status: taken

[admin_api_without_login]
expected: 403 because admin session is required
actual: status=403, success=False, message=admin permission required

[admin_login_default_demo_account]
expected: 200 in local demo; startup warns if default credentials are still used
actual: status=200, success=True, message=admin login successful
detail: {'data': {'admin_username': 'admin', 'is_admin': True}}

[admin_hash_only_messages]
expected: 200 and no content field because admin sees hash-only audit metadata
actual: status=200, success=True, message=admin fetched messages (hash only, no plaintext)
detail: {'contains_content_field': False}

[tamper_detection]
expected: VERIFY_FAIL with hash_mismatch_count > 0 after direct DB content modification
actual: status=200, success=False, message=integrity verification failed
detail: {'hash_mismatch_count': 1, 'root_match': False}

[plaintext_keyword_trace_blocked]
expected: 403 because admin plaintext keyword scanning is disabled
actual: status=403, success=False, message=管理员请勿读取聊天明文，关键词扫描溯源已禁用；请使用 /admin/trace 按订单号和溯源原因触发条件溯源。

```

图 40: attack_demo.py 运行输出（11 个场景全部通过）

S01-S02：身份与 Token 伪造 无登录态请求被 `user_required` 装饰器拦截返回 401；篡改 Token 末位后，`verify_token` 重算 HMAC-SM3 发现不匹配，返回 `valid=false`，证明在未知 K_{order} 下 Token 不可伪造。

S03：跨用户越权 后端从 session 获取当前用户 PID 与订单归属 PID 比对，不一致即返回 403，防止 Token 跨用户复用。

S04-S06：骑手接单 S04 验证未登录骑手被拒（401），S05 为基线（首次接单成功，状态 `created`→`taken`），S06 验证重复接单被状态机拦截（400）。S05 的成功是 S06 拒绝的前提，两者共同证明接单状态约束有效。

S07-S08：管理员访问 S07 验证无管理员 session 时返回 403 并写入审计日志（`action=TRACE_DENIED`），S08 为基线证明管理员认证流程正常。

S09:hash-only 审计 管理员消息接口返回数据仅含 `msg_id`、`role`、`message_hash` 和 `timestamp`，不含 `content` 字段，实现审计与隐私访问的接口级权限分离。

S10-S11：完整性与溯源 S10 验证篡改消息密文后完整性验证返回 `VERIFY_FAIL`（`hash_mismatch_count=1`，`root_match=False`），详见 5.4 节。S11 验证管理员无法绕过审批流程直接扫描明文，溯源接口返回 403，溯源必须经完整性校验和审批双重门槛。

5.4 日志篡改检测测试

本节验证 Merkle Tree 完整性机制对通信日志篡改的检测能力，覆盖修改密文、删除消息、插入伪造消息和顺序重排四种篡改类型。测试在数据库层面直接执行，模拟内部攻击者物理访问 SQLite 的场景。

首先建立正常通信场景：用户与骑手在同一订单房间发送多条消息，管理员生成 Merkle Root 快照。此时完整性验证返回 `VERIFY_OK`，所有消息哈希一致，当前 Root 与快照 Root 匹配。

随后执行篡改：攻击者直接修改数据库中某条消息的加密内容字段。由于消息哈希绑定原始明文，修改密文后解密结果与原哈希不匹配。完整性验证返回 `VERIFY_FAIL`，`hash_mismatch_count=1`，`root_match=False`。该场景的运行输出见 5.3 节图 40 中 `[tamper_detection]` 部分。四种篡改类型检测结果如表 14 所示。

表 14: 篡改类型与检测结果

篡改类型	Root 变化	mismatch_count	验证结果
修改消息密文	Root 变化	1	VERIFY_FAIL
删除消息记录	Root 变化	0	VERIFY_FAIL
插入伪造消息	Root 变化	0	VERIFY_FAIL
消息顺序重排	Root 变化	0	VERIFY_FAIL

其中“修改密文”已通过 `attack_demo` 脚本 S10 项实际验证，其余三种基于 Merkle Tree 机制分析：删除、插入和重排虽不产生逐条哈希不匹配，但均改变 Merkle Tree 结构导致 Root 值变化。

综合测试结论：（1）内容篡改可精确定位——系统不仅检测到 Root 不匹配，还能报告 `stored_hash` 与 `expected_hash` 的差异条目；（2）结构篡改可全局检测——删除、插入和重排通过 Root 比对检测到完整性异常；（3）三端一致性增强——攻击者需同时篡改三方存储的 Root 才能伪造一致结果，单方面篡改必然被暴露。上述结论与第三章安全性分析一致：SM3 抗碰撞性（碰撞复杂度 2^{128} ）保证字段修改后哈希以压倒性概率改变，Merkle Tree 级联特性保证叶子变化传导至 Root。

5.5 越权访问测试

本节验证系统三层访问控制边界的严格执行能力。部分场景与 5.3 节安全测试重叠（S03 跨用户越权、S07 管理员越权），此处补充骑手越权和审计日志细节。

用户越权：用户 alice 尝试查看非本人订单的消息历史，后端校验 session 中 user_id 与订单归属不一致，返回 403。

骑手越权：非接单骑手尝试进入订单房间发送消息或查看通信记录，后端校验 rider_id 不一致，返回 403。

管理员越权：未登录用户访问管理员审计接口，全部返回 403 并写入审计日志，如表 15 所示。

表 15: 越权访问审计日志记录

接口路径	状态码	审计 action	拒绝原因
/admin/messages/{OID}	403	TRACE_DENIED	not_admin
/admin/audit_logs/{OID}	403	TRACE_DENIED	not_admin
/admin/verify/{OID}	403	TRACE_DENIED	not_admin
/admin/orders	403	TRACE_DENIED	not_admin
/admin/trace (POST)	403	TRACE_DENIED	not_admin

溯源滥用阻断：管理员未经审批尝试明文关键词扫描，系统返回 403 阻断。当前溯源流程为审批式——用户/骑手提交溯源申请，管理员审批后按订单号自动完成溯源，管理员不能绕过审批直接扫描明文。

三层访问控制与 PID 后端托管、hash-only 审计共同实现第三章定义的目标五（可追责性）和目标七（隐私最小化）。管理员越权操作的审计记录使管理员自身行为也处于可追踪视野中，防止“谁来审计审计者”的信任困境。

5.6 性能测试

本节使用性能基准脚本对关键密码学操作和业务流程进行耗时测试。两组测试分别使用不同数据规模（第一组：10 用户、50 订单、1000 消息；第二组：20 用户、200 订单、6000 消息），结果如表 16 所示，终端运行输出如图 41 和图 42 所示。

表 16: 关键路径性能基准

操作	第一组 (50 订单, 1000 消息)					第二组 (200 订单, 6000 消息)				
	avg	P95	max	N	单位	avg	P95	max	N	单位
Token 生成	1.199	1.488	1.862	50	ms	1.181	1.421	1.888	200	ms
Token 验证	1.133	1.374	1.683	50	ms	1.130	1.379	4.007	200	ms
SM4 加解密	0.547	0.607	1.641	1000	ms	0.550	0.624	3.975	6000	ms
消息写入 + 哈希	0.454	0.582	1.355	1000	ms	0.461	0.577	4.242	6000	ms
Merkle 快照	19.90	21.29	21.76	50	ms	28.17	29.82	32.82	200	ms
完整性验证	37.62	39.09	40.69	50	ms	54.48	57.02	61.09	200	ms

```

[mic@LTD:~/FoodShield]$ python -m project.tools.performance_benchmark
Database initialized successfully.
FoodShield performance benchmark
database: /tmp/foodshield_perf_13a6c85efcbc49549085caea0fe3223cf.db
users=10, riders=10, orders=50, messages=1000
token_generate      count=50      avg=    1.199ms p95=    1.488ms max=    1.862ms
token_verify        count=50      avg=    1.133ms p95=    1.374ms max=    1.683ms
sm4_roundtrip       count=1000    avg=    0.547ms p95=    0.607ms max=    1.641ms
message_write       count=1000    avg=    0.454ms p95=    0.582ms max=    1.355ms
merkle_snapshot     count=50      avg=   19.898ms p95=   21.287ms max=   21.764ms
integrity_verify    count=50      avg=   37.622ms p95=   39.087ms max=   40.686ms

```

图 41: 第一组性能基准运行输出

```

[mic@LTD:~/FoodShield]$ uv run python -m project.tools.performance_benchmark --users 20 --orders-per-user 10 --messages-per-order 30
Database initialized successfully.
FoodShield performance benchmark
database: /tmp/foodshield_perf_744fd185b57646c7858c952861776990.db
users=20, riders=20, orders=200, messages=6000
token_generate      count=200     avg=    1.181ms p95=    1.421ms max=    1.888ms
token_verify        count=200     avg=    1.130ms p95=    1.379ms max=    4.007ms
sm4_roundtrip       count=6000    avg=    0.550ms p95=    0.624ms max=    3.975ms
message_write       count=6000    avg=    0.461ms p95=    0.577ms max=    4.242ms
merkle_snapshot     count=200     avg=   28.167ms p95=   29.822ms max=   32.822ms
integrity_verify    count=200     avg=   54.483ms p95=   57.017ms max=   61.091ms

```

图 42: 第二组性能基准运行输出

运行命令: 第一组 `python -m project.tools.performance_benchmark`; 第二组追加 `--users 20 --orders-per-user 10 --messages-per-order 30`。

性能分析

- 密码学操作耗时可控:** Token 生成与验证约 1.1–1.2 ms, 两组基本一致, 说明 HMAC-SM3 耗时不受数据规模影响。SM4 加解密约 0.55 ms, 消息写入 (含加密 + 哈希 + SQLite INSERT) 约 0.46 ms, 均满足实时通信要求。PID 生成同为 HMAC-SM3 操作, 耗时与 Token 相当。
- Merkle 线性增长:** 快照生成从 19.90 ms (20 条/订单) 增至 28.17 ms (30 条/订

单), 每条消息约 0.94–0.99 ms; 完整性验证从 37.62 ms 增至 54.48 ms, 每条约 1.82–1.88 ms, 均符合 $O(n)$ 理论预期。

3. **瓶颈识别**: 完整性验证是最耗时操作 (需解密全部消息并重算哈希), 但触发频率低 (仅管理员主动验证或溯源前置校验时触发), 不影响日常通信。日常关键路径 (Token 验证 + SM4 加解密 + 消息写入) 总耗时约 2 ms。

5.7 对比分析

本节将 FoodShield 与现有外卖平台通信方案、普通 WebSocket 通信系统以及学术匿名通信方案进行安全特性对比, 如表 17 所示。

表 17: FoodShield 与现有方案安全特性对比

安全属性	FoodShield	美团/饿了么	普通 WebSocket	群签名方案
目标一匿名性	PID 强匿名	虚拟号码 (弱)	无匿名	签名匿名
目标二认证性	HMAC-SM3	无凭证认证	无认证	群签名认证
目标三机密性	SM4-CBC 加密	传输加密 + 明文	传输加密 + 明文	加密 (可选)
目标四完整性	SM3+Merkle	无验证	无验证	无 Merkle
目标五可溯源	条件溯源	实名查询	不可溯源	群管理打开
目标六抗滥用	验证码 + 限流	无机制	无机制	群成员管理
目标七隐私最小化	hash-only	全量可见	全量可见	条件打开
系统原型	已实现	商业运行	通用实现	仅理论

与美团/饿了么对比: 现有平台采用虚拟号码和脱敏显示, 但存在三个不足: 虚拟号码仅保护手机号, 骑手仍可通过订单备注和地址推断用户习惯; 管理员可无条件查看通信明文, 缺乏权限分离; 通信记录明文存储, 无完整性验证。FoodShield 通过 PID (跨订单不可链接)、SM4 加密存储、hash-only 审计和 Merkle 完整性验证系统性改进了上述问题。

与普通 WebSocket 对比: 普通 WebSocket 仅依赖传输层加密, 攻击者获取订单号即可进入通信通道 (无 Token 认证), 管理员可查看全部明文, 日志可被任意篡改。FoodShield 在通信层之上叠加了 Token 认证、PID 匿名、SM4 加密和 Merkle 验证, 将普通即时通信升级为可认证、可审计的安全通信。

与群签名方案对比: 群签名在理论上提供更强的匿名性 (不可链接性), 但存在三个实际限制: 签名/验证涉及多次双线性映射运算, 单次耗时数十毫秒, 不适合高频低延迟的外卖通信; 缺乏 Merkle 等完整性保护机制; 未见针对外卖场景的系统原型。FoodShield 虽匿名性理论强度稍逊, 但在计算效率、完整性和系统可用性方面更具优势。

综合评价：FoodShield 的核心差异化在于“可控匿名”模型——日常通信中对骑手和攻击者实现 PID 强匿名，同时在投诉驱动和审批约束下保留条件溯源能力。该模型介于“完全实名不可匿”与“完全匿名不可治”之间，在外卖订单通信场景中实现了匿名性、认证性、机密性、完整性、可溯源性和隐私最小化的多目标平衡。

第六章 系统创新性与应用价值

本章从功能展示和创新性两个角度对 FoodShield 系统进行总结。与普通外卖订单聊天系统相比，本作品并不只关注用户与骑手之间能否完成消息发送，而是将匿名身份、订单认证、消息保护、日志完整性验证、条件溯源和管理员审计统一纳入外卖订单通信流程中，形成了面向真实业务场景的隐私认证与可信审计闭环。系统的创新性主要体现在场景选择、机制组合、安全模型、工程闭环和应用价值五个方面。

6.1 贴近生活的高频应用场景

现有外卖平台通常已经具备用户与骑手之间的即时通信能力，但其核心目标主要是提升配送效率，而非系统性解决通信过程中的隐私保护与证据可信问题。在实际外卖场景中，用户与骑手之间的通信往往伴随订单号、配送地址、联系方式、备注信息和聊天内容等多类敏感信息。如果系统缺少精细化的权限边界，骑手端或管理员端可能接触到超出配送所需范围的信息；如果系统缺少可信审计机制，平台在处理纠纷、骚扰投诉或恶意争议时又难以证明通信记录是否保持原始状态。

FoodShield 的场景创新在于：将外卖订单通信从普通业务聊天场景提升为一个兼具隐私保护、身份认证和可信审计需求的安全场景。系统以订单为基本单位组织通信、认证和审计流程，使订单号不仅是业务编号，同时也是通信房间、认证凭证、日志记录和条件溯源的统一索引。用户与骑手的日常通信在匿名保护下完成，异常场景下则通过订单号触发受控溯源流程，从而避免了传统方案中“直接实名导致隐私暴露”和“完全匿名导致无法管理”两类极端问题。

此外，本作品选择外卖平台这一高频、真实、贴近日常生活的应用场景，能够较直观地展示信息安全技术在生活服务平台中的实际价值。相比单纯的算法演示或通用匿名聊天系统，本作品更强调安全机制与具体业务流程之间的结合，使系统创新具有更强的可解释性和展示效果。

6.2 安全机制与订单流程的结合

FoodShield 的机制创新主要体现在多种安全机制的组合使用。系统并未将匿名身份、订单认证、加密存储、哈希审计和条件溯源作为彼此独立的功能点，而是围绕外卖订单通信流程进行集成设计，使每个安全机制都服务于明确的业务环节。

首先，系统设计了基于 PID 的匿名身份机制。用户注册后由后端生成匿名身份标识 PID，PID 作为系统内部身份参与订单绑定和通信记录，而用户真实身份不直

接暴露给骑手端。骑手端仅能看到订单相关信息和通信内容，无法通过普通业务接口获取用户真实身份或 PID，从而降低跨订单关联和身份暴露风险。

其次，系统设计了基于 HMAC-SM3 的订单 Token 认证机制。用户创建订单后，后端将订单号、匿名身份、时间戳等信息绑定生成 Token。用户进入订单通信房间前必须提交订单号和 Token，服务器验证通过后才允许其加入对应订单房间。该机制使用户能够在不暴露真实身份的情况下证明自己是合法订单参与方，避免仅凭订单号即可进入通信通道的越权风险。

再次，系统将 SM4 加密存储、SM3 消息摘要和 Merkle Tree 完整性验证结合起来。通信消息在写入数据库前进行加密处理，管理员端默认只查看消息哈希、订单摘要和 Merkle Root 等审计信息，而不是直接查看消息明文。系统以消息摘要作为 Merkle Tree 叶子节点，通过 Root 快照比对检测消息修改、删除、插入和重排等篡改行为，使聊天记录从普通数据库记录转变为可验证的审计对象。

最后，系统引入三端摘要一致性增强机制，将服务端、用户端和骑手端的 Merkle Root 进行比对。该机制提高了攻击者单独篡改平台侧日志和快照的难度，使日志完整性验证不再完全依赖单一服务端存储结果，从而进一步增强了通信记录的可信性。

6.3 日常匿名、异常可追责的安全模型

FoodShield 采用的是“日常匿名、异常可溯源”的可控匿名安全模型。该模型不同于完全实名模式，也不同于完全匿名模式。完全实名模式虽然便于平台追责，但会增加用户真实身份、联系方式和订单偏好暴露的风险；完全匿名模式虽然保护隐私，但在发生恶意骚扰、虚假投诉或配送纠纷时又可能导致责任主体难以确认。FoodShield 在两者之间建立了一个受控平衡点。

在日常通信状态下，系统通过 PID 匿名身份、订单级通信房间和字段最小化展示机制，限制骑手端和管理员端能够看到的信息范围。骑手端不展示用户真实身份，不展示用户 PID，不展示用户端私有备注和标签；管理员端默认不展示通信明文，而是以订单摘要、消息哈希、Merkle Root 和审计日志作为主要查看对象。这种设计降低了普通业务流程中的身份暴露风险。

在异常处理状态下，系统允许用户端和骑手端基于订单号提交溯源申请，管理员经过身份认证后处理申请，并将相关操作写入审计日志。溯源入口从 PID 调整为订单号，使溯源流程更符合真实外卖平台中的投诉和纠纷处理逻辑，也降低了 PID 被直接滥用查询的风险。管理员的查询、验证和溯源行为均被纳入审计范围，从而避免审计权限本身成为新的隐私风险。

因此，FoodShield 的安全模型创新不只是“隐藏身份”，而是将匿名性、认证性、完整性、可追责性和可审计性统一到同一套订单生命周期中，实现了隐私保护与平台管理之间的动态平衡。

6.4 多端协同的工程完整性

本作品实现了用户端、骑手端和管理员端的完整系统闭环。用户端支持注册登录、创建订单、Token 验证、订单通信、订单历史查看和溯源申请；骑手端支持订单查看、接单、进入订单通信房间和异常申请；管理员端支持登录认证、订单审计、消息摘要查看、Merkle Root 快照生成、完整性验证、三端摘要一致性验证和条件溯源处理。

从业务流程看，系统形成了如下闭环：



图 43: 系统安全闭环流程图

该闭环体现了本作品的工程完整性。系统不是仅实现一个单点功能，例如只生成 PID、只验证 Token 或只计算 Merkle Root，而是将多个安全机制嵌入到完整业务流程中。用户下单后的每一步操作都与后端安全控制相关联：订单创建绑定匿名身份，进入通信房间需要 Token 验证，消息发送触发加密存储和哈希计算，管理员验证依赖 Merkle Root 快照，异常处理通过订单号进行受控溯源。

这种闭环设计提高了作品的可展示性和可答辩性。评审可以从用户端、骑手端和管理员端分别观察系统运行效果，也可以通过正常通信、Token 篡改、日志篡改、

权限校验和条件溯源等场景验证系统安全机制是否有效。相比仅停留在理论方案层面的设计，本作品提供了可运行、可交互、可演示的系统原型。

6.5 实际应用价值与推广前景

FoodShield 的应用价值主要体现在外卖平台隐私保护、即时配送安全通信、平台纠纷处理和电子证据可信化四个方面。

第一，在外卖平台隐私保护方面，系统通过匿名身份、字段最小化展示和消息加密存储机制，减少用户真实身份和敏感订单信息在普通配送流程中的暴露。骑手端只获取完成配送所需的信息，管理员端默认只查看审计摘要而非通信明文，符合个人信息保护中“最小必要”的设计思想。

第二，在即时配送安全通信方面，FoodShield 的核心机制不仅适用于餐饮外卖，也可以扩展至快递配送、跑腿代购、即时零售、生鲜配送和药品配送等场景。这些场景均存在用户与配送人员之间的短时通信需求，同时也存在身份泄露、骚扰投诉和订单纠纷等风险。系统中的 PID、Token、Merkle 审计和条件溯源模块可以作为可复用组件迁移到其他订单通信平台。

第三，在平台纠纷处理方面，系统为平台提供了兼顾隐私保护和责任追踪的技术路径。日常通信中，用户身份得到保护；异常事件发生时，平台可以基于订单号、审计日志和完整性验证结果进行责任确认。该机制有助于提高平台处理恶意骚扰、虚假投诉和配送争议的效率与可信度。

第四，在电子证据可信化方面，系统通过消息哈希、Merkle Root 快照和完整性验证机制，使通信记录具备可验证性。对于订单纠纷处理而言，聊天记录不再只是数据库中的普通文本，而是能够通过密码学摘要和 Root 比对证明其是否被篡改的审计对象。这一思路也可推广到电商客服、在线医疗问诊、网约服务沟通和共享经济平台等需要保存可信通信记录的场景。

综上，FoodShield 的创新价值并不仅仅局限于某一个密码算法的使用，而在于将国密算法、匿名身份、订单认证、实时通信、可信审计和条件溯源有机结合，构建了一个面向外卖订单通信场景的隐私认证与可信审计系统。该系统兼具工程可实现性、场景针对性和安全展示价值，能够为生活服务平台中的隐私保护与可信管理提供参考。

第七章 总结

7.1 作品总结

本文围绕外卖订单通信场景中的身份暴露、订单隐私泄露、聊天记录篡改和纠纷追责困难等问题，设计并实现了 FoodShield——外卖订单隐私认证与可信审计系统。系统以“日常匿名、异常可溯源”为核心设计思想，将匿名身份、订单认证、实时通信、消息保护、日志完整性验证和管理员审计机制统一到外卖订单通信流程中，形成了较为完整的安全通信闭环。

在身份保护方面，系统通过 PID 匿名身份机制将用户真实身份与通信身份分离，使骑手端在日常通信过程中无法直接获取用户真实身份或 PID。在订单认证方面，系统基于 HMAC-SM3 构造订单 Token，将订单号、匿名身份和时间戳等信息进行绑定，保证只有合法订单参与方才能进入对应通信房间。在消息保护方面，系统对通信内容进行 SM4 加密存储，并使用 SM3 生成消息摘要，管理员端默认不展示通信明文，从而降低敏感信息泄露风险。

在可信审计方面，系统将消息摘要组织为 Merkle Tree，并通过 Merkle Root 快照实现通信日志完整性验证。当消息内容、消息数量或消息顺序被篡改时，重新计算得到的 Root 将与历史快照不一致，从而能够检测日志篡改行为。在异常处理方面，系统支持用户端和骑手端基于订单号提交溯源申请，管理员经过权限认证后执行条件溯源，并将关键操作写入审计日志，实现了隐私保护与平台管理之间的平衡。

通过系统实现与功能展示可以看出，FoodShield 不只是一个普通的订单聊天系统，而是将密码学机制与外卖订单业务流程相结合的隐私认证与可信审计原型系统。系统实现了用户端、骑手端和管理员端的多端协同，覆盖了注册、下单、认证、通信、审计、验证和溯源等完整流程，具有较好的工程完整性和展示价值。

7.2 现有不足

FoodShield 当前仍处于竞赛原型阶段，在以下方面存在不足：

第一，系统基于 SQLite 和单机后端完成核心流程验证，未在高并发通信、分布式存储和大规模订单数据处理场景下进行充分测试，系统吞吐量和并发能力有待评估。

第二，PID 映射仍由平台单点保存，存在平台单点信任风险。虽然系统通过审计日志和权限控制约束了管理员的溯源行为，但尚未引入盲签名、零知识证明或多方审批等去信任化机制。

第三，SM4-CBC 提供机密性保护，完整性由应用层 SM3 和 Merkle Tree 负责，两者独立运作。生产环境中可考虑采用认证加密模式（如 GCM），将加密与完整性验证合为一步，降低实现复杂度。

第四，系统当前以 HTTP 运行，通信链路未启用 TLS/WSS 加密。虽然竞赛演示环境下不影响安全机制验证，但在真实部署中需结合传输层安全措施。

第五，条件溯源的审批流程较为简化，尚未实现多级审批、多人联合授权和溯源理由模板等细粒度管控机制，管理员权限的约束仍依赖审计日志而非技术强制。

7.3 未来工作

围绕当前原型系统已经实现的功能闭环，FoodShield 后续工作将主要面向应用场景扩展、安全机制增强、可信审计提升、条件溯源规范化和工程化部署五个方向展开。当前系统已经验证了匿名身份、订单认证、安全通信、日志完整性验证、条件溯源和可信审计机制在外卖订单通信场景中的可行性，但在面向真实平台环境时，仍需要进一步提升系统的可扩展性、鲁棒性和安全强度。

首先，在应用场景方面，FoodShield 的设计思想并不局限于餐饮外卖场景。系统中基于订单号的身份认证、基于 PID 的匿名通信、基于 Merkle Root 的日志完整性验证以及基于管理员权限的条件溯源机制，均可以迁移到其他即时服务平台中。例如，在快递配送、跑腿代购、生鲜配送、药品配送、网约维修、电商客服等场景中，用户与服务人员之间都存在短时通信、身份隔离、责任确认和通信记录存证等需求。因此，FoodShield 后续可以进一步扩展为面向多类订单服务场景的通用隐私通信与可信审计框架。

其次，在安全机制方面，系统可以继续增强订单级安全能力。未来可以围绕订单生命周期构建更加细粒度的安全策略，例如为不同订单生成独立的通信密钥和匿名身份标识，进一步降低跨订单关联风险；同时，可以将 Token 认证、消息摘要、Merkle 快照和审计日志进行更紧密的联动，使订单从创建、通信、验证到溯源的全过程都具备更强的一致性和可验证性。通过这些机制扩展，系统能够进一步提升对伪造订单、越权访问、日志篡改和恶意溯源等行为的防护能力。

再次，在可信审计方面，FoodShield 可以继续提升审计结果的外部可信性。当前系统已经通过消息哈希和 Merkle Tree 实现了通信日志完整性验证，后续可以进一步引入可信时间戳、独立审计节点或联盟链式存证机制，将关键 Merkle Root 快照写入更难被单点篡改的外部环境中。这样可以进一步增强平台在纠纷处理、投诉核验和责任确认过程中的证据可信度，使通信记录不仅能够在系统内部验证，也能

够在更广泛的可信环境中被复核。

此外，在条件溯源方面，系统可以进一步完善分级授权与审批机制。未来可以将溯源操作划分为不同等级，例如普通异常申请、严重投诉处理、平台安全事件处理等，并为不同等级配置不同的审批流程、管理员权限和审计要求。对于高敏感度的溯源请求，可以采用多管理员联合审批、溯源理由模板、操作留痕和结果回执等方式，使溯源过程更加规范、透明和可追踪。这样既能保障异常场景下的责任追踪能力，也能避免管理员权限被滥用。

最后，在工程化应用方面，FoodShield 可以进一步向可部署、可扩展的真实平台原型演进。后续可以继续完善前端交互体验、接口文档、自动化测试、异常处理、日志管理和并发通信能力，并结合真实外卖订单流程设计更加完整的业务接口。随着系统工程化程度的提升，FoodShield 可以从竞赛原型逐步发展为适用于生活服务平台的隐私保护与可信审计组件，为平台在用户隐私保护、配送纠纷处理和安全合规建设方面提供技术参考。

综上，FoodShield 的后续发展方向并不是单纯补充某一项功能，而是围绕“匿名通信、订单认证、可信审计、条件溯源”这一核心思路，进一步拓展应用场景、增强安全机制、提升审计可信性和完善工程化能力。随着即时配送和本地生活服务平台的持续发展，此类兼顾隐私保护与责任追踪的安全通信机制具有较好的应用前景。

参考文献

- [1] 全国人民代表大会常务委员会. 中华人民共和国密码法 [Z]. 主席令第三十五号, 2019-10-26.
- [2] 全国人民代表大会常务委员会. 中华人民共和国个人信息保护法 [Z]. 2021-08-20.
- [3] 全国人民代表大会常务委员会. 中华人民共和国数据安全法 [Z]. 2021-06-10.
- [4] 国务院. “十四五”数字经济发展规划 [Z]. 国发〔2021〕29号, 2021-12.
- [5] GB/T 32905-2016. 信息安全技术 SM3 密码杂凑算法 [S]. 北京: 中国标准出版社, 2016.
- [6] GB/T 32907-2016. 信息安全技术 SM4 分组密码算法 [S]. 北京: 中国标准出版社, 2016.
- [7] GB/T 32918-2016. 信息安全技术 SM2 椭圆曲线公钥密码算法 [S]. 北京: 中国标准出版社, 2016.
- [8] Bellare M, Canetti R, Krawczyk H. Keyed-Hashing for Message Authentication[S]. RFC 2104, IETF, 1997.
- [9] Fette I, Melnikov A. The WebSocket Protocol[S]. RFC 6455, IETF, 2011.
- [10] Merkle R C. A Certified Digital Signature[C]//Advances in Cryptology – CRYPTO ’89 Proceedings, LNCS vol. 435. Springer, 1989: 218–238.
- [11] 中国互联网络信息中心. 第 55 次中国互联网络发展状况统计报告 [R]. 2025.
- [12] 中国饭店协会. 2024 年中国餐饮业年度报告 [R]. 2024.
- [13] 韩丹东. 商品明细是隐私, 外卖员不应知晓 [N]. 法治日报, 2025-05-08(04).
- [14] 美团规则中心. 消费者权益保护 [EB/OL]. <https://rules-center.meituan.com/customer-rights/1>.
- [15] Lin X, Sun X, Ho P H, et al. GSIS: A Secure and Privacy-Preserving Protocol for Vehicular Communications[J]. IEEE Transactions on Vehicular Technology, 2007, 56(6): 3442–3456.

- [16] Raya M, Hubaux J P. Securing Vehicular Ad Hoc Networks[J]. Journal of Computer Security, 2007, 15(1): 39–68.
- [17] Wasef A, Shen X. Efficient Privacy-Preserving Authentication with Conditional Traceability for VANETs[C]//IEEE Global Telecommunications Conference (GLOBECOM). IEEE, 2009: 1–6.
- [18] Stinson D R. 密码学原理与实践 [M]. 冯登国, 等译. 3 版. 北京: 电子工业出版社, 2016.
- [19] Stallings W. Cryptography and Network Security: Principles and Practice[M]. 8th ed. Pearson, 2020.
- [20] 国家密码管理局. 商用密码管理条例 [Z]. 2023-04-27.

A 附录

1.1 源代码仓库

本项目已开源至 GitHub¹，仓库中包含完整的系统源代码、攻击演示脚本、性能测试脚本和说明文档。

1.2 项目目录结构

```
FoodShield/
|-- project/
|   |-- crypto/           # SM3/SM4/HMAC、PID、Token、Merkle
|   |-- database/        # SQLite schema 与访问封装
|   |-- frontend/        # 用户端、骑手端、管理员端页面
|   |-- server/          # Flask 主程序与审计逻辑
|   |-- tools/           # attack_demo.py, performance_benchmark.py
|   |-- integration_demo.py
|-- requirements.txt
|-- pyproject.toml
|-- foodshield.db
```

1.3 运行方式

```
pip install -r requirements.txt
python -m project.server.app
```

启动后访问 <http://127.0.0.1:5000/>，分别打开用户端、骑手端和管理员端页面。

¹<https://github.com/hxhdhcjmet/FoodShield.git>